ROSETTA

Orquestador de cumplimiento continuo

MEMORIA TÉCNICA · PRÁCTICA 3

1. Portada

CAMPO	VALOR
Modalidad	Proyecto individual, autorizado por el equipo docente
Autor	Michael Jordan Telleria Guadalajara
Fecha de entrega	16/10/2026
Repositorio	https://github.com/MichaelJTG/Rosetta
Tag de entrega	`v1.0-practica3`
Vídeo	https://drive.google.com/file/d/1tBNHq--ITta3mDXcef4TIyqi_SYZcIqk/view?usp=sharing (unos 25 minutos; la parte 2 empieza en el 20:40)
Despliegue	Evaluación en local con Docker siguiendo el README. Existe un despliegue en un servidor propio que a fecha de entrega no está operativo (apdo. 10.1)

El vídeo dura unos 25 minutos, por encima de los 20 recomendados, porque cubre los 37 requisitos; la matriz indica el minuto de cada uno. La parte final (errores y seguridad, calidad y cierre) se grabó el 16/10 como continuación y se indica en pantalla.

Índice

1. Portada 1
2. Resumen ejecutivo 3
3. Punto de partida 4
4. Requisitos 6
5. Arquitectura y decisiones técnicas 9
6. Funcionalidades implementadas 12
7. Seguridad del producto 23
8. Pruebas y evidencias 28
9. Matriz de trazabilidad 32
10. Limitaciones y trabajo futuro 34
11. Reparto del trabajo 35
12. Uso de herramientas de IA 36
13. Anexos 38

2. Resumen ejecutivo

ROSETTA es una plataforma de cumplimiento normativo continuo que recibe hallazgos técnicos arbitrarios —resultados de escáneres de seguridad, alertas de SIEM, observaciones de auditor— y los traduce automáticamente a controles normativos de los marcos ISO 27001:2022, ENS RD 311/2022, NIS2, DORA, RGPD, NIST CSF 2.0 y PCI-DSS 4.0 mediante un «Traductor Simbiótico» (RAG + LLM con tool-use forzado).

Destinatarios: responsable de cumplimiento, CISO, auditor técnico que necesita transformar evidencia operativa en evidencia de auditoría sin trabajo manual.

Estado de entrega: producto funcional en local con Docker, que es como se evalúa (README y Anexo A); el despliegue en un servidor propio de la P1 no está operativo a fecha de entrega (apdo. 10.1). Cumple 23 de 25 ítems RF (2 en ◐ con condiciones: RF-06, RF-09) y 11 de 12 ítems RNF (1 en ◐: RNF-09, solo la primera traducción con el modelo en frío excede 30 s). Eval cuantitativa ejecutada: F1 = 0,2648 (correspondencia ENS→ISO, 73 casos, 0 % alucinación) y F1 = 0,2813 (hallazgos técnicos, 16 casos, 0 % alucinación). CI verde.

Cifras clave (2026-10-10):

Indicador	Valor
Tests automatizados	556 (pytest) + 1 E2E con Playwright fuera de la CI
Cobertura	85 % global · 92 % en `core/`
Endpoints HTTP	31 REST + 1 WebSocket
Paneles en el dashboard	16 (requisito base: 10)
Marcos normativos	7
Fragmentos en corpus	229
F1 eval correspondencia	0,2648
F1 eval hallazgos	0,2813
Alucinación (controles inventados)	0 % en ambos modos
Build desde cero	165 s, imagen 3,85 GB
CVEs sin parche	4 (chromadb HTTP server, no aplicable a ROSETTA)

3. Punto de partida

3.1 Resumen de la Práctica 1

ROSETTA en la Práctica 1 era ya una plataforma funcional, desplegada en un servidor propio. El informe entregado el **25 de mayo de 2026** describía los siguientes componentes operativos (43 commits, entre 2026-04-18 y 2026-05-30):

- **API REST + WebSocket:** FastAPI con 15 endpoints REST y 1 WebSocket de progreso en tiempo real para el Modo Auditoría.
- **Dashboard SPA de 10 paneles:** servido desde el propio backend; los paneles Inicio, Traducir, Cumplimiento, Auditoría, PDF, Blue Team, Grafo, Drift, Copilot y Hallazgos estaban operativos.
- **Corpus normativo en 7 marcos:** ISO 27001:2022, ENS RD 311/2022, NIS2, DORA, RGPD, NIST CSF 2.0 y PCI-DSS 4.0 indexados en ChromaDB con embeddings multilingües.
- **Traductor Simbiótico con tool-use forzado:** RAG + LLM con esquema Pydantic; latencia típica <4 s con Claude API.
- **Arquitectura multi-agente:** scheduler Soundwave + agente Validador implementados.
- **Adaptadores Red Team y Blue Team:** Nuclei, Nmap, Amass (Red) y Wazuh (Blue) orquestados sin fork.
- **Grafo Neo4j + degradación grácil:** correlación activo→control; modo memoria si Neo4j no disponible.
- **Persistencia SQLite:** historial de hallazgos append-only (SessionStore).
- **CLI Typer:** comandos `translate`, `load-corpus`, `version`, `dossier`, `scan`, `detect-drift`.
- **Gate CI/CD:** `POST /analyze-diff` + GitHub Action para bloquear PRs que incumplen controles.
- **Auth JWT + HTTP Basic + rate limiting** (slowapi): Bearer como método principal.
- **Docker:** imagen y despliegue con Docker Compose en un servidor propio, detrás de nginx con HTTPS. Ese servidor usaba Groq (`llama-3.3-70b-versatile`) a través del proveedor compatible con OpenAI, no Claude (apdo. 5.4). A fecha de entrega de la P3 el despliegue no está operativo (apdo. 10.1).
- **24 ficheros de test:** ruff + mypy --strict activos; CI configurada pero rota desde el 2026-05-20.
- **Herramientas de IA en P1:** Gemini, para planificar y ordenar ideas. Claude Code, para la programación; parte del código lo escribió el autor. Además, los conocimientos del Grado Superior en DAM.

Varias de estas afirmaciones del informe de la P1 resultaron inexactas al revisarlas en la P3: Amass tenía adaptador pero no estaba instalado ni conectado al Modo Auditoría; la API tenía 18 rutas REST y no 15 (hoy son 31); y la latencia «<4 s» con Claude nunca se midió. El detalle está en el apartado 5.4 (Cambios respecto a la P1).

En resumen: ROSETTA en P1 era una aplicación completa y desplegada, con API, dashboard, RAG, multi-agente, Docker y 7 marcos normativos. La P3 no partió de cero; partió de una base funcional con bloqueos concretos documentados en la tabla del apartado 3.3.

3.2 Feedback recibido

Nota de la P1: 9/10. Feedback oral breve del equipo docente; la plataforma no recoge comentarios escritos.

3.3 Clasificación inicial al comienzo de P3

> Estado medido el 2026-10-04 (fase 0 de P3), usando las categorías del enunciado (§3).

Categoría	Ítems	Notas
Cumplido	RF-01, RF-02, RF-03, RF-07, RF-08, RF-10, RF-11, RF-12, RF-13, RF-15, RF-16, RF-17, RF-19, RNF-03, RNF-07	Funcionales desde P1; mantenidos en P3
Parcial	RF-04 (sin endpoint de descarga), RF-06 (Nmap/Nuclei no en Docker, sin demo WebSocket), RF-09 (sin benchmark), RF-14 (sin demo en CI externa), RF-18 (CLI al 0 % cobertura), RNF-02 (77 % vs. 80 %), RNF-04 (ruff falla en 2 tests), RNF-06 (credencial en historial), RNF-08 (hostname esquivaba bloqueo), RNF-10 (CI roja desde 2026-05-20), RNF-12 (pip-audit auditaba herramienta, no proyecto)	Funcionalidad escrita pero con bloqueos activos identificados en la fase 0
Pendiente	RF-05 (sin demo E2E verifiable), RF-20a..RF-20f (endpoints escritos sin tests completos), RNF-01 (Neo4j caído: sin tests de API), RNF-05 (instalación limpia no verificada), RNF-09 (latencia no medida), RNF-11 (cabeceras HTTP: sin middleware dedicado)	No iniciados o sin evidencia verificable antes del bloque B
A revisar	RF-15 (JWT presente; rate limiting no demostrado aisladamente)	Implementados pero sin evidencia aislada de cumplimiento
Descartado	Adaptadores Shodan/HIBP, integración Jira/ServiceNow, multi-tenant SaaS	Fuera del alcance del enunciado o desproporcionados para el plazo

Estado de la CI al inicio de P3: en rojo desde el 2026-05-20 («Install uv» fallaba por versión hardcodeada). Los tests pasaban en local pero no en GitHub Actions.

4. Requisitos

4.1 Lista completa de RF con estado final

ID	Descripción	Estado
RF-01	Traducir hallazgo técnico → controles normativos con cita y justificación	✔
RF-02	Multi-marco: ISO 27001:2022, ENS 2022, NIS2, DORA, RGPD, NIST CSF 2.0, PCI-DSS 4.0	✔
RF-03	Pipeline RAG con ChromaDB, embeddings multilingües	✔
RF-04	Dosier de auditoría Markdown + PDF descargable	✔
RF-05	Ingesta de PDF de auditoría humana, extracción de hallazgos	✔
RF-06	Modo Auditoría Red Team (Nmap, Nuclei) con progreso WebSocket	◐ Parcial — implementado; demo solo contra laboratorio local
RF-07	Integración Blue Team — ingesta alertas Wazuh JSON/CSV	✔
RF-08	Correlación Red↔Blue de hallazgos	✔
RF-09	Arquitectura multi-agente: Soundwave + Validador	◐ Parcial — Validador opcional con `"validar": true`; Soundwave sin endpoint de producción
RF-10	Copilot normativo con citas y nivel de confianza	✔
RF-11	Detección de procedure drift	✔
RF-12	Dashboard SPA con 10 paneles servido desde el backend (P1); ampliado a 16 paneles en P3	✔
RF-13	API REST documentada con Swagger UI	✔
RF-14	Gate CI/CD (analizar diff de PR, bloquear incumplimientos)	✔
RF-15	Auth JWT Bearer + HTTP Basic fallback	✔
RF-16	Historial de hallazgos persistente en SQLite (append-only)	✔
RF-17	Grafo de correlación Neo4j + vis.js con degradación grácil	✔
RF-18	CLI Typer: translate, load-corpus, version, dossier, scan, detect-drift	✔
RF-19	Panel de cumplimiento por marco	✔
RF-20a	Catálogo de controles con detalle y estado	✔
RF-20b	Roadmap de vulnerabilidades	✔
RF-20c	Panel de evidencias	✔

ID	Descripción	Estado
RF-20d	Gap analysis por marco	✔
RF-20e	Plan director automático	✔
RF-20f	Análisis de riesgos	✔

4.2 Lista completa de RNF con estado final

ID	Descripción	Estado
RNF-01	Degradación grácil sin Neo4j	✔
RNF-02	Cobertura ≥ 80 % en core	✔ 92 % en `core/` ; 85 % global (solo `pdf_ingestion.py` por debajo: 76 %)
RNF-03	Sin errores `mypy --strict`	✔ 0 errores en 56 ficheros
RNF-04	Sin violaciones `ruff`	✔
RNF-05	Instalable desde cero con `uv sync` + `docker compose up`	✔
RNF-06	Cero secretos en el repositorio	✔
RNF-07	Rate limiting en endpoints sensibles	✔
RNF-08	Validación de alcance en Modo Auditoría	✔
RNF-09	Tiempo del Traductor < 30 s	◐ p50=6,95 s, p95=11,0 s; 1/20 excede el umbral: la primera llamada, en frío (59,9 s)
RNF-10	Build CI reproducible (lockfile)	✔
RNF-11	Cabeceras de seguridad HTTP	✔
RNF-12	Audit CVE dependencias en CI	✔

4.3 Requisitos modificados y descartados respecto a P1

RF/RNF	Versión P1	Versión P3	Justificación
RF-12	10 paneles (base)	16 paneles (6 nuevos: Controles, Roadmap, Evidencias, Gap, Plan Director, Riesgos)	Compliance dashboard amplía el requisito base
RF-15	HTTP Basic	JWT Bearer como principal + Basic como fallback	Seguridad: JWT con expiración; Basic solo para clientes heredados y `/docs`
RF-20	Un único ítem «panel avanzado»	Desglosado en RF-20a…RF-20f	Necesario para trazabilidad en matriz y vídeo
RF-21 (P1)	Adaptadores Shodan/HIBP	Movido a trabajo futuro	Solo existe enum y mapa de controles; sin adaptador implementado

RF/RNF	Versión P1	Versión P3	Justificación
RNF-02	≥ 80 % «estimado»	77 % medido el 2026-10-04 (~503 tests); 85 % global y 92 % en `core/` el 2026-10-10 (556 tests)	Medición real con el comando de la CI
RNF-09	Sin umbral definido	< 30 s con LLM; benchmark de 20 llamadas (`eval/bench_latency.py`)	Benchmark cuantificado con Ollama qwen2.5:14b
RNF-10	`uv.lock` en `.gitignore`	Lockfile versionado	Build reproducible exigible por el enunciado

4.4 Mejoras introducidas respecto a P1

- **Corpus ENS completo** (73 medidas de RD 311/2022 con campo `iso_27001_2022`): necesario para el eval cuantitativo.
- **Harness de evaluación cuantitativa** con ground truth externo (SoA del docente): F1, hallucination rate y discrepancy rate.
- **Laboratorio Red Team local** (Nmap 7.95 + Nuclei 3.11.1 en la imagen Docker): el Modo Auditoría funciona sin dependencias externas.
- **Descarga de dosier** (MD y PDF) desde el dashboard: completaba RF-04 que en P1 no tenía endpoint de descarga.
- **Endurecimiento de seguridad** (bloques B-1 a B-11): CSP, XSS, path traversal, rate limiting, cabeceras, PDF magic bytes, gate SSRF.
- **CI corregida y reproducible** (RNF-10, RNF-12): lockfile versionado, `setup-uv v7.6.0` , `pip-audit` sobre `uv.lock` .

5. Arquitectura y decisiones técnicas

5.1 Diagrama de componentes

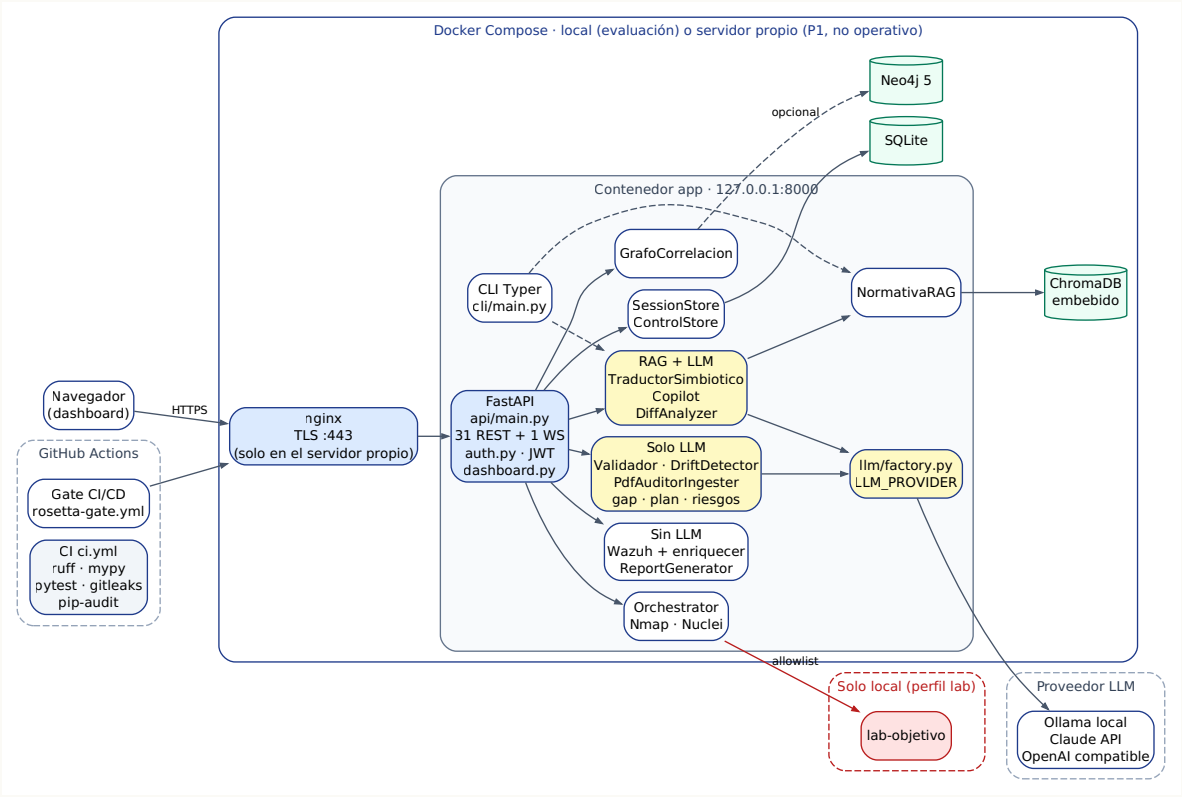


Figura 1. Diagrama de componentes de ROSETTA, comprobado contra los imports de `src/rosetta/` y los workflows de `.github/` : nginx con TLS delante de la app en 127.0.0.1:8000 (solo en el servidor propio de la P1; en local se accede directamente a 127.0.0.1:8000), núcleo, persistencia, proveedor LLM elegido con `LLM_PROVIDER` y CI. Fuente: `docs/memoria/diagramas/componentes.dot` . (RF-01, RF-02, RF-03)

5.2 Tecnologías y motivo de elección

Tecnología	Versión	Motivo
Python 3.11 / 3.12	3.11+	Madurez tipo-segura; `asyncio` nativo en FastAPI
FastAPI	≥0.115	API asíncrona, OpenAPI automático, Pydantic v2 integrado
Pydantic v2	≥2.7	Validación con `model_validator` y `pattern=` ; fuerza tool-use estructurado
ChromaDB 1.5.9	embedded	Vector store embebido; sin servidor extra; persistencia en volumen Docker
Neo4j 5	5.x	Grafo de correlación activo↔control; degradación grácil cuando no disponible
Ollama	0.4.x	LLM local sin API key; desarrollo económico y soberanía de datos
Claude API (Anthropic SDK)	≥0.25	Proveedor en la nube opcional (`LLM_PROVIDER=claude`); forzado de tool-use fiable

Tecnología	Versión	Motivo
reportlab	≥4.4	PDF del dosier de auditoría (`core/report_generator.py`); puro Python, sin GTK
WeasyPrint	≥70	Solo genera el PDF de esta memoria (`scripts/build_memoria.py`); no interviene en el producto
Nmap 7.95 + Nuclei 3.11.1	—	Herramientas MIT/Apache orquestadas por CLI (ADR-001)
`uv`	≥0.4	Gestor de paquetes con lockfile determinista
`ruff + mypy --strict`	—	Calidad de código obligatoria en pre-commit y CI
structlog	—	Logs estructurados; audit trail con timestamp e IP real
slowapi	—	Rate limiting decorador sobre FastAPI

5.3 Decisiones de arquitectura clave

ADR-001 — Orquestación sobre fork/reimplementación (2026-04-18, aprobado)

El escáner Red Team y el SIEM Blue Team se consumen vía CLI/API sin modificar su código. Motivo: evita contaminación copyleft (Wazuh es GPL); recibimos mejoras upstream automáticamente; el valor diferencial de ROSETTA está en el Traductor, no en los sensores. Consecuencia negativa aceptada: dependencia de binarios externos en el host.

ADR-002 — Abstracción del proveedor LLM detrás de `LLMClient` Protocol (2026-04-18, aprobado)

`TraductorSimbiótico` recibe el cliente LLM por inyección de dependencia. Tres implementaciones: `ClaudeClient` (API de Anthropic), `OllamaClient` (local y on-premise; el de la evaluación), `OpenAIClient` (cualquier API compatible con OpenAI; el despliegue de la P1 lo usaba con Groq). Seleccionado por `LLM_PROVIDER`. Motivo: iteración local sin coste, soberanía de datos para clientes ENS/NIS2, testabilidad mediante mocks del Protocol. Consecuencia negativa: tres implementaciones que mantener.

Tool-use forzado con esquema Pydantic. El LLM no puede devolver texto libre: la llamada usa la función `traducir_hallazgo` con un `model_json_schema` derivado de `ResultadoTraduccion`. Si la respuesta no cumple el esquema, se rechaza con 422. Esto reduce alucinación de controles al 0 % en todos los evals ejecutados.

Mapeo determinista del panel de evidencias. El panel de evidencias y el panel de cumplimiento se calculan a partir del historial SQLite sin invocar al LLM, garantizando determinismo y velocidad.

Degradación grácil sin Neo4j. Al arrancar, la API crea `GrafoCorrelacion` (`core/graph.py`), que llama a `verify_connectivity()` con timeouts de 5 s. Si Neo4j no responde, se registra `neo4j_unavailable_api_continues`, la API sigue sin grafo y los endpoints de estado responden con los datos en memoria de la sesión. El usuario no ve ningún error.

ChromaDB embebido sin servidor. `PersistentClient` escribe directamente en el volumen `rosetta_chroma`. No se lanza ningún servidor Chroma. Limitación documentada: si `load-corpus` se ejecuta desde un proceso externo mientras la API está en marcha, la API debe reiniciarse para ver los nuevos fragmentos.

5.4 Cambios respecto a P1

Los cambios de arquitectura principales introducidos en P3:

- Corpus ENS enriquecido con las 73 medidas de RD 311/2022 y campo `iso_27001_2022` para el eval.
- Harness de evaluación (`eval/run_eval.py`) con ground truth externo.
- Nmap y Nuclei integrados en la imagen Docker; validación de alcance (DNS + `is_global` + allowlist).
- Lockfile `uv.lock` versionado; CI corregida (verde desde 2026-10-04).
- Bloque de seguridad B-1…B-11: path traversal, XSS, rate limiting, cabeceras HTTP, PDF limits, SSRF gate, CVE dependencias.
- `GET /reports/download/{filename}` y botón en dashboard para descarga de dossier.
- Campo `"validar": true` en el cuerpo de `POST /translate` para invocar el Validador (segundo agente LLM de crítica).
- 6 paneles adicionales al dashboard: Controles, Roadmap, Evidencias, Gap, Plan Director, Riesgos.

Afirmaciones del informe de la P1 que resultaron inexactas (revisión D-4 de la bitácora, comprobada contra el código):

Informe de la P1	Realidad comprobada
«FastAPI expone 15 endpoints REST y un WebSocket»	En la versión entregada (commit `992bf42`, 2026-05-20) había 18 rutas REST y 1 WebSocket. Hoy son 30 operaciones REST en OpenAPI (29 rutas), más `/dashboard`, que queda fuera del esquema, y 1 WebSocket.
Adaptadores Red Team: Nuclei, Nmap y Amass	Amass tiene adaptador (`adapters/red/amass.py`), pero no está instalado en la imagen ni conectado al orquestador. El Modo Auditoría solo usa Nmap y Nuclei.
Latencia típica «<4 s por traducción» con Claude	Nunca se midió. El benchmark de RNF-09 (P3) es con Ollama `qwen2.5:14b`: p50 = 6,95 s en 20 llamadas.
Tasa de alucinación como métrica ya medida	No existía harness de evaluación; se creó en la P3 (`eval/run_eval.py`, apartado 8.5).
LLM «Claude (producción)»	El servidor estaba configurado con Groq (`llama-3.3-70b-versatile`) a través del proveedor compatible con OpenAI (`LLM_PROVIDER=openai`; `.env` del servidor revisado el 2026-10-14).
WeasyPrint como generador del PDF	El PDF del dosier lo genera reportlab (`core/report_generator.py`).

6. Funcionalidades implementadas

6.1 Traducción de hallazgo a controles normativos (`POST /translate`)

Qué hace: recibe un `HallazgoMaestro` (origen, activo detectado, evidencia, vector de ataque, severidad, marco) y devuelve los controles del marco incumplidos, cada uno con cita literal del corpus y justificación. La IA no puede inventar controles: tool-use forzado con esquema Pydantic.

Cómo se usa: desde el panel «Traducir» del dashboard, la CLI (`rosetta translate`) o directamente por API con token JWT. El campo opcional `"validar": true` del cuerpo invoca el Validador para una segunda opinión.

Requisitos: RF-01 (traducción), RF-02 (multi-marco), RF-03 (RAG).

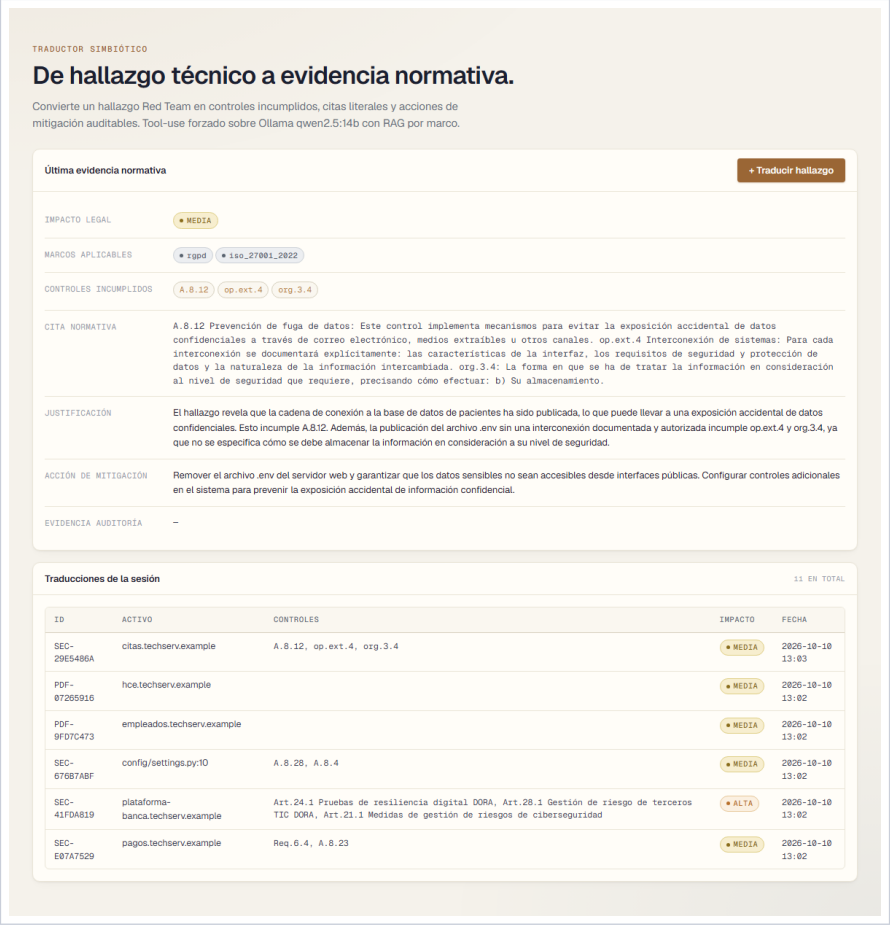


Figura 2. Panel «Traducir»: hallazgo ficticio de TechServ (fichero `.env` publicado en `citas.techserv.example`) traducido contra ENS e ISO 27001, con cita, justificación y mitigación. (RF-01, RF-02, RF-03)

IA vs. determinismo: el Traductor usa el LLM para la justificación y la selección de controles; el RAG (ChromaDB) aporta los fragmentos relevantes del corpus de forma determinista.

6.2 Pipeline RAG sobre corpus normativo (ChromaDB)

Qué hace: indexa los 7 marcos normativos como fragmentos semánticos. En cada traducción, recupera los N fragmentos más relevantes para el hallazgo y el marco elegido, que se inyectan en el prompt del LLM.

Estado: 229 fragmentos indexados con `rosetta load-corpus all corpus` el 2026-10-10 (ISO 27001: 93 · ENS: 73 · NIS2: 15 · NIST CSF: 14 · DORA: 13 · PCI-DSS: 12 · RGPD: 9). La instalación limpia del 2026-10-04 indexó 197, antes de ampliar el corpus ENS. Modelo de embeddings: `paraphrase-multilingual-MiniLM-L12-v2` (multilingüe español/inglés). Cobertura 100 %.

Requisitos: RF-03.

IA vs. determinismo: la recuperación es determinista (coseno de similitud sobre embeddings precomputados); el LLM solo procesa los fragmentos recuperados.

6.3 Generación de dosier de auditoría (RF-04)

Qué hace: genera un informe Markdown o PDF con todos los hallazgos de la sesión, los controles incumplidos y las recomendaciones. Descargable desde el dashboard (botón «Generar dosier») o por API (`GET /reports/download/{filename}`).

Cómo se usa: panel «Hallazgos» → botón «Generar dosier» → descarga MD o PDF.

Requisitos: RF-04.

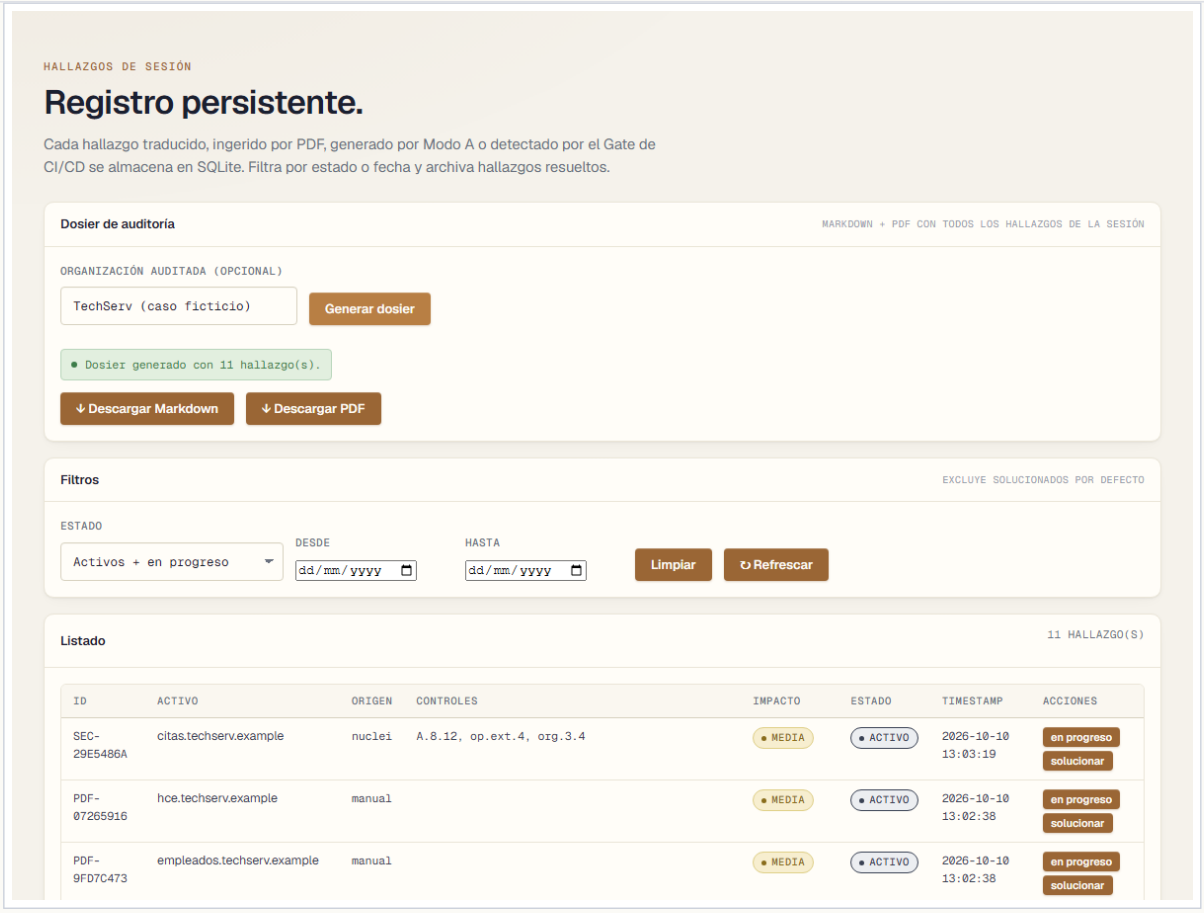


Figura 3. Panel «Hallazgos»: dosier generado para TechServ (caso ficticio) con descarga en Markdown y PDF, y listado persistente de hallazgos. (RF-04, RF-16)

IA vs. determinismo: el informe se genera de forma determinista a partir del historial SQLite.

6.4 Ingesta de PDF de auditoría humana (RF-05)

Qué hace: extrae hallazgos de un PDF de informe de auditoría existente (pdfplumber) con fallback a visión LLM para páginas escaneadas. Devuelve una lista de `HallazgoMaestro` listos para traducir.

Cómo se usa: panel «PDF» → subir informe → hallazgos extraídos. O `POST /ingest/pdf`.

Requisitos: RF-05.



Figura 4. Panel «Ingesta PDF»: informe de auditoría ficticio de TechServ; 2 hallazgos extraídos en modo texto. (RF-05)

IA vs. determinismo: la extracción de texto (pdfplumber) es determinista; la identificación de los hallazgos la hace siempre el LLM con tool-use forzado (`extraer_hallazgos`). La visión LLM (pypdfium2) solo se usa si la página es una imagen escaneada.

6.5 Modo Auditoría Red Team (RF-06)

Qué hace: lanza un escaneo Nmap + Nuclei contra el objetivo declarado, emite progreso en tiempo real por WebSocket y almacena los hallazgos en el historial. Valida que el objetivo no sea IP privada, localhost o fuera de la allowlist del servidor.

Cómo se usa: panel «Auditoría» → introducir objetivo → iniciar escaneo. O `POST /audit/start` + `WS /audit/ws/{audit_id}`.

Requisitos: RF-06, RNF-08.

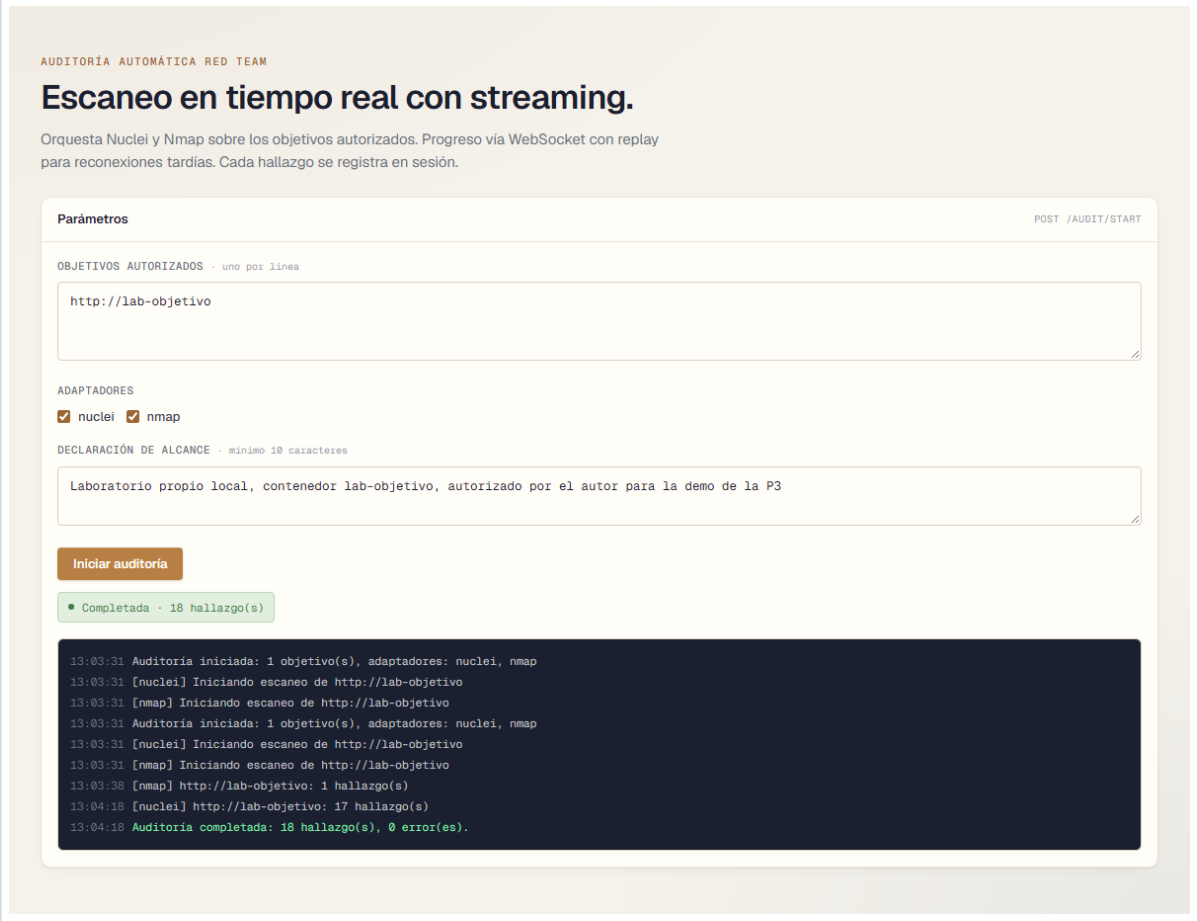


Figura 5. Panel «Modo Auditoría» contra el laboratorio local (`http://lab-objetivo`) con Nuclei y Nmap y progreso por WebSocket. La ejecución del 2026-10-10 terminó con 18 hallazgos y 0 errores. (RF-06, RNF-08)

Resultado contra el laboratorio (2026-10-10): 18 hallazgos en unos 45 s: 17 de Nuclei (el `.env` publicado, con severidad alta; la versión de nginx; las cabeceras de seguridad ausentes) y 1 de Nmap (puerto 80/tcp). El listado de `/backups/` no lo detecta ninguna plantilla pública de Nuclei. Las plantillas se filtran con `ROSETTA_NUCLEI_TAGS` (por defecto `exposure,misconfig,tech`) y `ROSETTA_NUCLEI_SEVERITY`, y Nuclei corre sin interactsh: no envía tráfico a servidores externos.

Limitación: la demo en el vídeo se realiza contra el laboratorio local (objetivo sin IPs reales para proteger a terceros).

IA vs. determinismo: Nmap y Nuclei son deterministas; los hallazgos resultantes se traducen después por el Traductor (que sí usa LLM).

6.6 Blue Team — Wazuh (RF-07, RF-08)

Qué hace: recibe alertas Wazuh en formato JSON o CSV y las normaliza a `DatosBlue`. El módulo `BlueEnrichment` cruza los hallazgos Red Team con las alertas Blue por activo, enriqueciendo el contexto para el Traductor.

Cómo se usa: `POST /blue/ingest` con JSON de alertas. Panel «Blue Team» muestra los hallazgos correlacionados.

Requisitos: RF-07, RF-08.

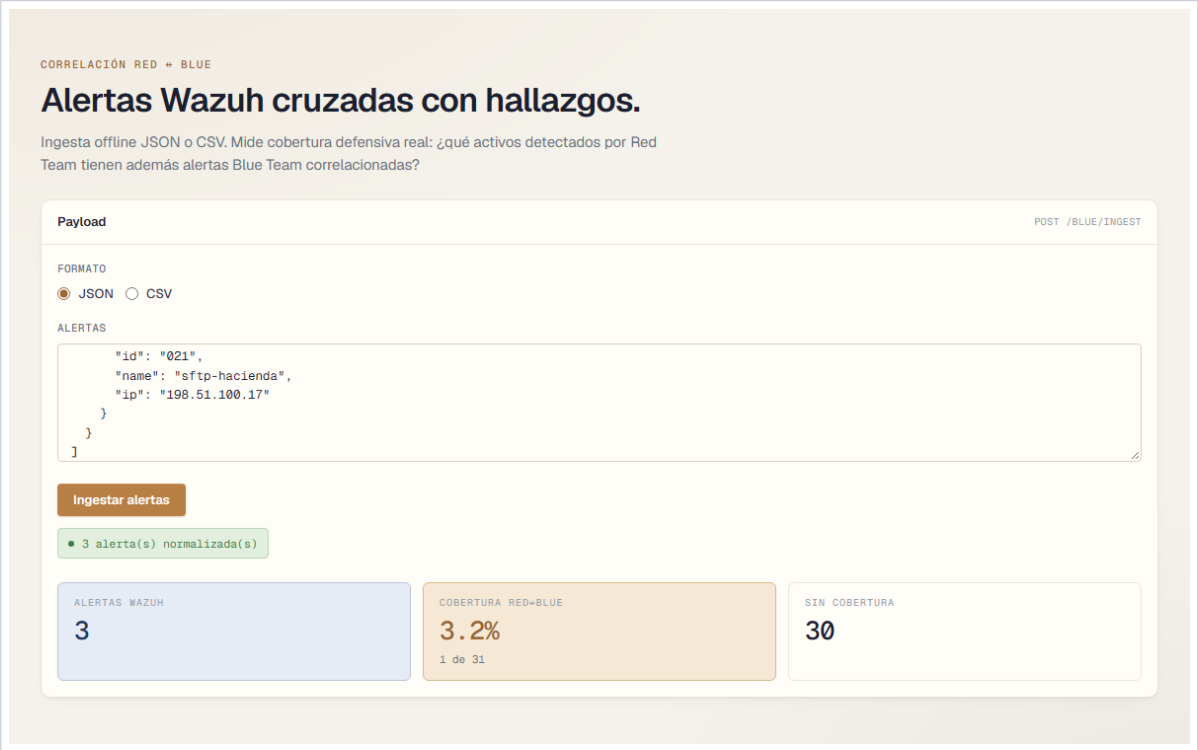


Figura 6. Panel «Blue Team»: 3 alertas Wazuh ficticias normalizadas y cobertura Red↔Blue de la sesión (1 de 31 activos, 3,2 %). (RF-07, RF-08)

6.7 Arquitectura multi-agente — Validador (RF-09)

Qué hace: el Validador es un segundo agente LLM que revisa la traducción del Traductor y la aprueba o la rechaza; no la modifica. Se activa con el campo `"validar": true` del cuerpo de `POST /translate`. El scheduler Soundwave existe (`agents/soundwave.py`), pero ningún endpoint lo usa.

Requisitos: RF-09.

Estado: Validador operativo; Soundwave sin endpoint de producción. Sobre los 73 casos de correspondencia, precisión de rechazo @F1<0,5 = 0,82 y F1 medio de 0,36 en las traducciones aprobadas frente a 0,17 en las rechazadas (apdo. 8.5).

6.8 Copilot normativo (RF-10)

Qué hace: responde preguntas en lenguaje natural sobre los marcos normativos, citando el control relevante y un nivel de confianza. Usa RAG sobre el mismo corpus que el Traductor.

Cómo se usa: panel «Copilot» → escribir pregunta → respuesta con cita y `confianza`. La `confianza` no es una probabilidad calibrada: vale `min(1, fragmentos recuperados / 5)`.

Requisitos: RF-10.

Con Ollama, el prompt del Copilot (unos 5100 tokens con los 7 marcos) no cabe en la ventana por defecto de Ollama; ROSETTA pide `OLLAMA_NUM_CTX` (16384 por defecto) en cada llamada.



Figura 7. Panel «Copilot»: pregunta sobre el doble factor en la VPN de TechServ (caso ficticio); la respuesta cita la medida `op.acc.6` del ENS. (RF-10)

6.9 Procedure Drift (RF-11)

Qué hace: compara un procedimiento escrito con el comportamiento observado por los sensores y cuantifica la divergencia (`drift_score`).

Cómo se usa: `POST /drift/analyze`. Panel «Drift» muestra el score y las diferencias detectadas.

Requisitos: RF-11.



Figura 8. Panel «Procedure Drift» (captura del 16/10/2026): procedimiento ficticio PRO-IAM-001 frente a tres observaciones; drift de severidad ALTA, fragmento afectado y redacción propuesta. Los controles los propone el LLM: el detector de drift no usa el RAG ni los comprueba contra el corpus, y `A.9.2` es un identificador de ISO 27001:2013, no de la versión 2022. (RF-11)

6.10 Dashboard SPA de 16 paneles (RF-12)

Qué hace: interfaz web completa servida desde `GET /dashboard`. En P1 tenía 10 paneles; P3 añade 6 paneles del compliance dashboard (Controles, Roadmap de vulnerabilidades, Evidencias, Pre-Gap Análisis, Plan Director, Análisis de Riesgos).

Requisitos: RF-12 (superado: requisito 10 paneles, implementados 16).

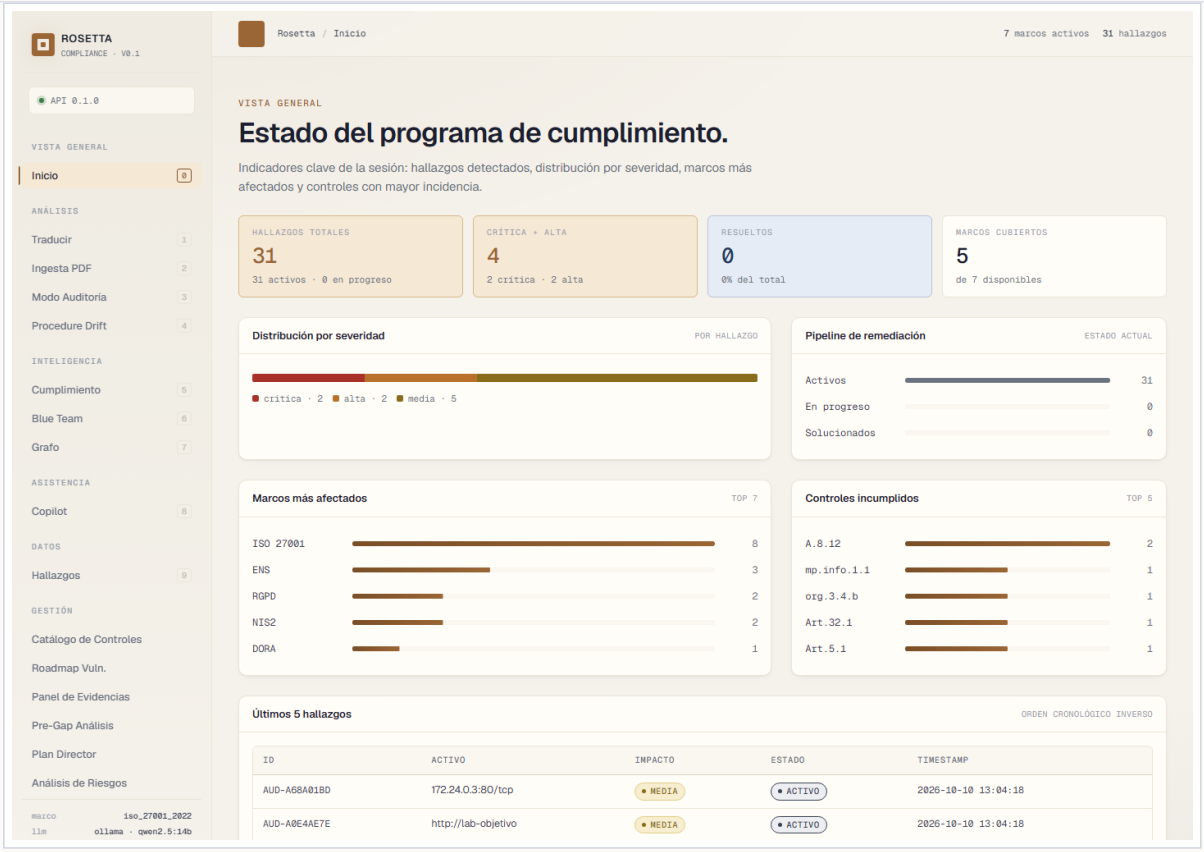


Figura 9. Panel «Inicio» del dashboard con la navegación lateral de los 16 paneles y los indicadores de la sesión. (RF-12)

6.11 Auth + rate limiting (RF-15)

Qué hace: `POST /auth/login` devuelve un JWT Bearer. Los endpoints protegidos requieren `Authorization: Bearer <token>`. HTTP Basic sigue disponible como fallback. Rate limiting (slowapi): 5 logins por minuto por IP; el sexto recibe 429.

Requisitos: RF-15, RNF-07.

6.12 Grafo de correlación Neo4j + vis.js (RF-17)

Qué hace: persiste la relación activo→control en Neo4j y la visualiza con vis.js. Si Neo4j no está disponible, el grafo funciona en modo memoria (degradación grácil).

Requisitos: RF-17, RNF-01.

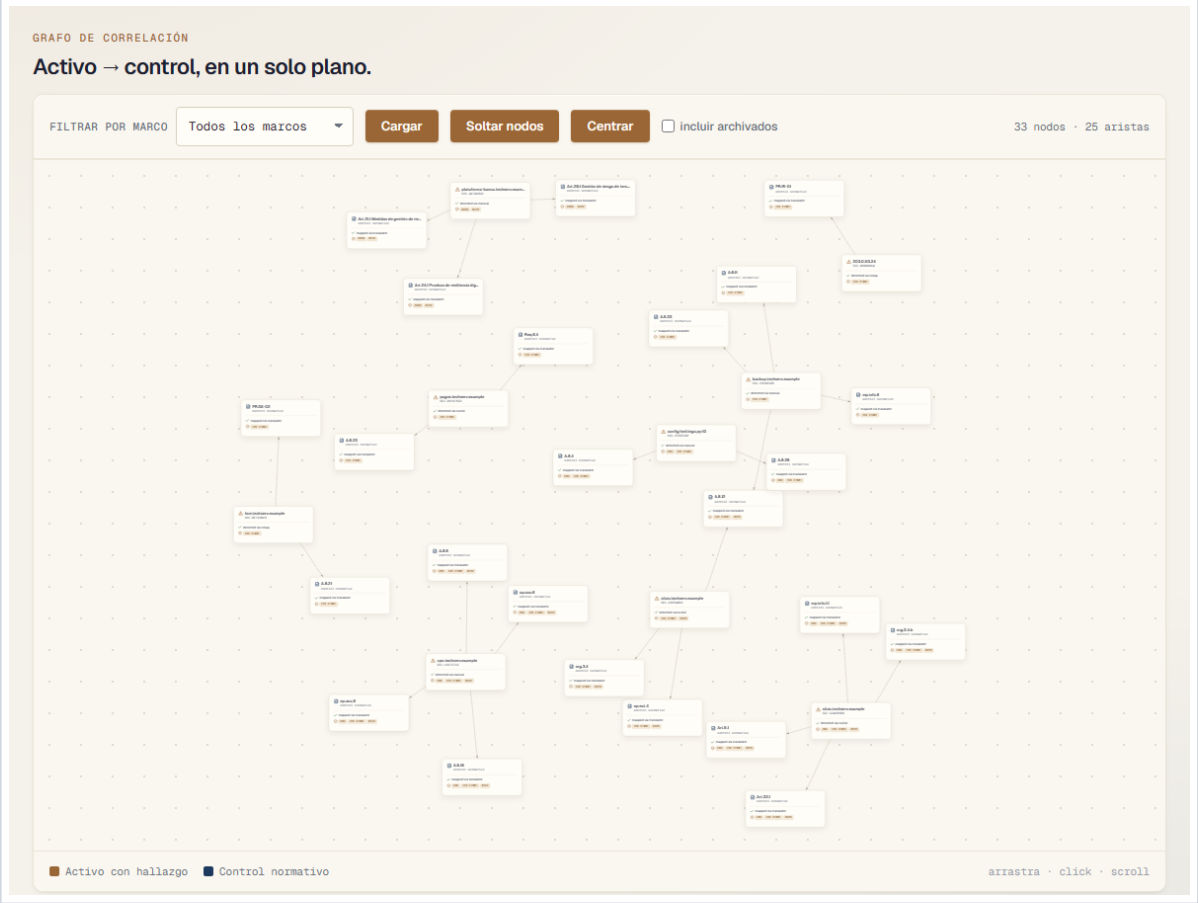


Figura 10. Panel «Grafo»: 33 nodos y 25 aristas activo→control de la sesión, leídos de Neo4j y dibujados con vis.js. (RF-17)

6.13 CLI Typer (RF-18)

Qué hace: línea de comandos con 6 comandos: `translate`, `load-corpus`, `version`, `dossier`, `scan`, `detect-drift`.

Cómo se usa: `rosetta version` → semver; `rosetta load-corpus all corpus/` → corpus indexado.

Requisitos: RF-18.

6.14 Compliance dashboard — 6 paneles adicionales (RF-19, RF-20a...RF-20f)

Qué hace: el grupo de paneles de cumplimiento ofrece vistas agregadas del estado normativo:

Panel	Endpoint	Qué muestra
Cumplimiento	`GET /compliance/state/{marco}`	Hallazgos por marco, controles más incumplidos y severidad
Catálogo de Controles	`GET /controls/{marco}` · `GET /controls/{marco}/{control_id}`	Catálogo con estado y detalle de un control con hallazgos asociados
Roadmap Vuln.	`GET /vuln-roadmap`	Hallazgos con trazabilidad temporal

Panel	Endpoint	Qué muestra
Panel de Evidencias	`GET /evidence-panel`	Evidencias ligadas a controles
Pre-Gap Análisis	`POST /gap-analysis/{marco}`	Brechas con severidad estimada
Plan Director	`POST /plan-director/{marco}`	Plan de acciones ordenado por prioridad
Análisis de Riesgos	`POST /risk-analysis`	Activos con scoring de riesgo

Requisitos: RF-19, RF-20a...RF-20f.

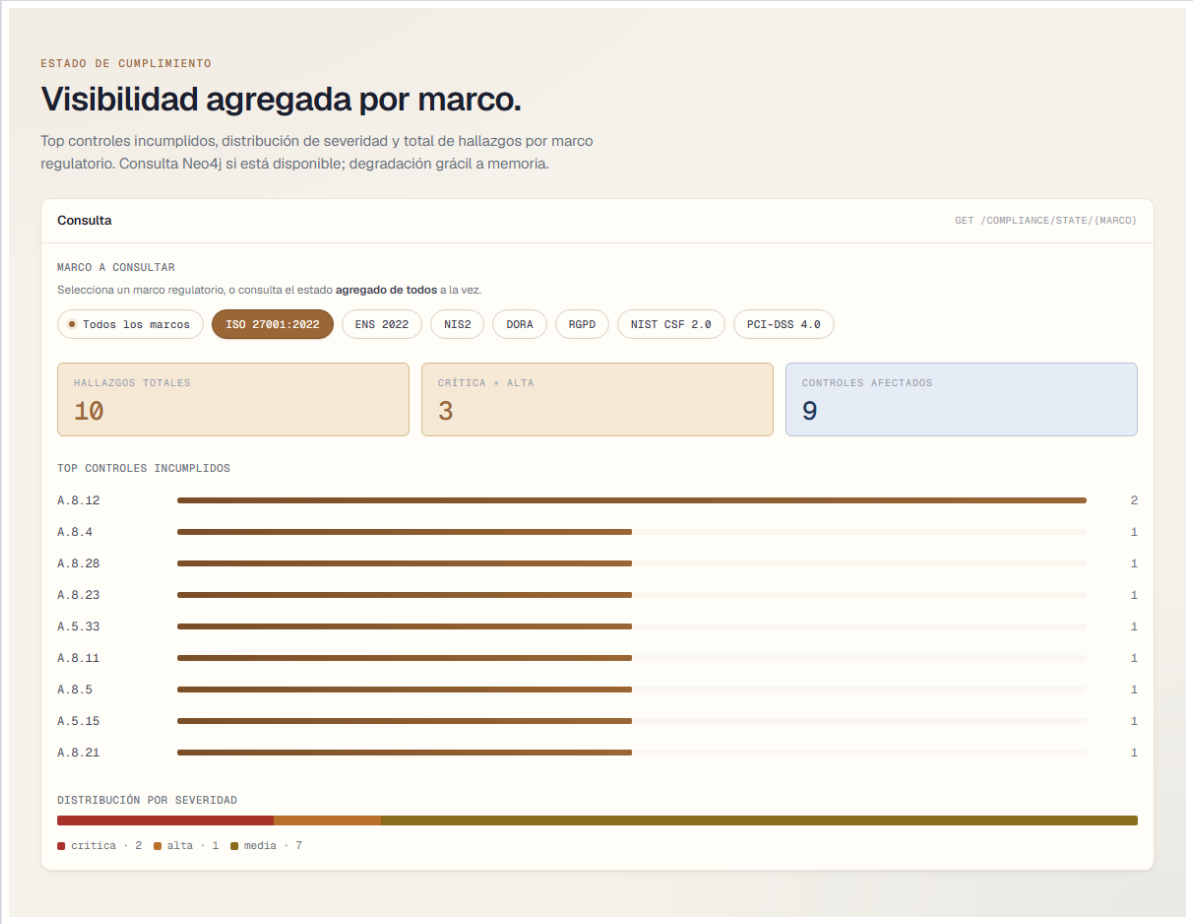


Figura 11. Panel «Cumplimiento» filtrado por ISO 27001: solo controles del Anexo A, con su recuento y la distribución por severidad. (RF-19)

6.15 Gate CI/CD (RF-14)

Qué hace: analiza el diff de un PR y devuelve `decision=block` si el cambio introduce código que incumple un control. El GitHub Action `.github/workflows/rosetta-gate.yml` integra este gate en el workflow de revisión.

Requisitos: RF-14.

6.16 OpenAPI / Swagger UI (RF-13)

Qué hace: FastAPI genera automáticamente el esquema en `GET /openapi.json`: 29 rutas y 30 operaciones (todas las REST salvo `/dashboard`, excluida del esquema). Swagger UI se sirve en `GET /docs` y ReDoc en `GET /redoc`, con una CSP propia que permite sus recursos de cdn.jsdelivr.net; el resto de rutas mantiene la CSP estricta de B-3.

Requisitos: RF-13.

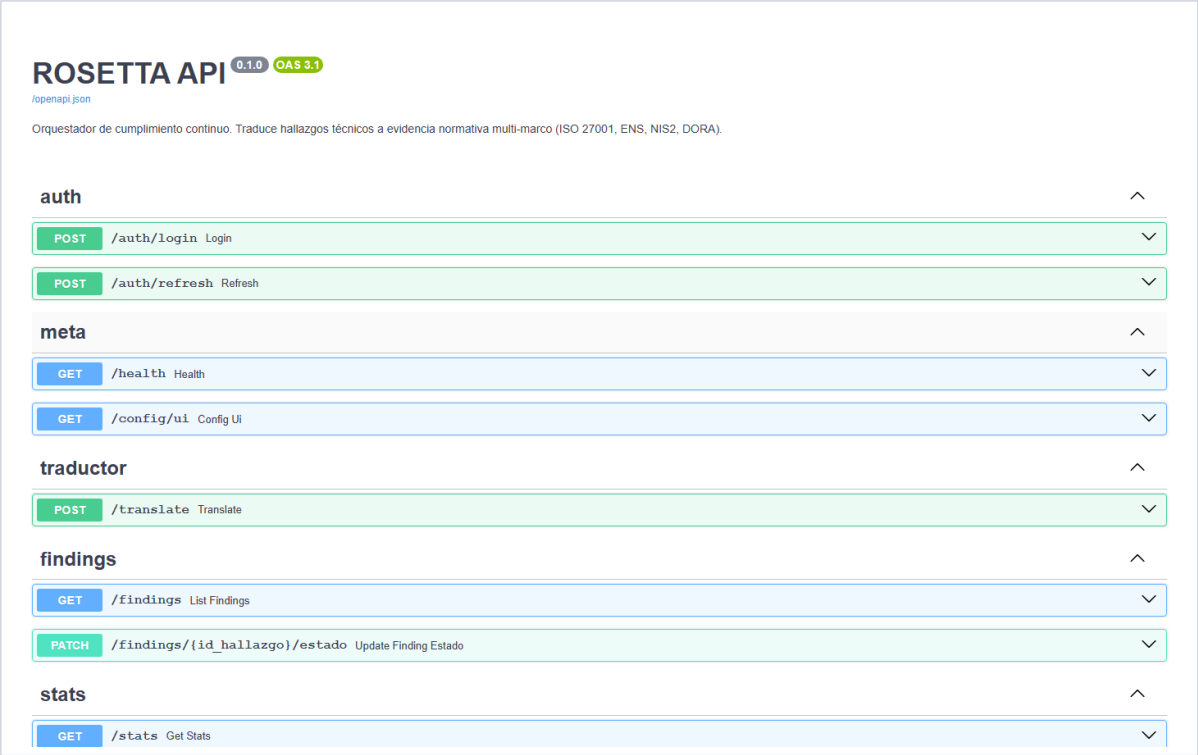


Figura 12. Swagger UI en `/docs` con las operaciones de la API agrupadas por etiqueta. (RF-13)

7. Seguridad del producto

7.1 Bloques de trabajo de seguridad completados en P3

Bloque	Descripción	Commit
B-1	Path traversal en descarga de informes (`/reports/download`). Vídeo 21:41 (en directo)	`5ca3ca8`
B-2	XSS via salida del LLM: `esc()` en todos los `innerHTML` con datos LLM/ usuario	`5ca3ca8`
B-3	Cabeceras de seguridad HTTP: CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy; CSP propia para `/docs` y `/redoc`	`c552729` · `c78a0c7`
B-4	Rate limiting de `slowapi` + validación de proxies confiables (solo loopback/ `172.18.0.1/32`)	`5cf5aac` · `f06f726`
B-5	JWT refresh + ROSETTA_USERS_EXTRA desde entorno (no hardcodeado)	`5cf5aac`

Bloque	Descripción	Commit
B-6	Mensajes de error genéricos en 500; stack trace solo en log	`5ca3ca8`
B-7	SSRF gate en Modo Auditoría: validación DNS + `is_global()` + allowlist	`810812d`
B-8	Scope validation (11 tests de alcance): IPs privadas, localhost, hostname resolution	`810812d`
B-9	PDF magic bytes + límite 20 MB + límite de páginas (4 tests). Vídeo 21:38 (en directo)	`5ca3ca8`
B-10	CVE dependencias en CI: 9 tandas de actualización; R-01 documentado	`2e7792d` … `9a611d7`
B-11	Trivy image scan	No implementado (fuera del sprint)

7.2 Modelo de amenazas STRIDE

S — Spoofing

ID	Amenaza	Control	Riesgo residual
S-1	Falsificación de JWT	Firma HS256 con `ROSETTA_JWT_SECRET` ≥ 32 chars	Bajo
S-2	Reutilización de token caducado	`exp` claim obligatorio; validación en cada request	Bajo
S-3	Credenciales débiles en `.env`	`.env.example` fuerza contraseñas ≥ 12 chars	Medio — sin validación de complejidad en runtime

T — Tampering

ID	Amenaza	Control	Riesgo residual
T-1	Modificación del corpus ChromaDB en volumen	Volumen `rosetta_chroma` sin puertos expuestos; acceso solo vía API	Bajo
T-2	Modificación del historial SQLite	Solo la API puede escribir en `session_store`; acceso por contenedor	Bajo
T-3	Path traversal en descarga de informes	`pathlib.Path.resolve()` + restricción a directorio de trabajo (B-1)	Bajo
T-4	Inyección de código en diff analizado	DiffAnalyzer procesa texto; no ejecuta el diff	Bajo

R — Repudiation

ID	Amenaza	Control	Riesgo residual
R-1	Ausencia de rastro de quién tradujo qué	`structlog` registra IP + usuario en cada traducción	Medio — logs en contenedor; sin audit log externo
R-2	Borrado de historial SQLite	Historial append-only; sin endpoint de borrado	Bajo

I — Information Disclosure

ID	Amenaza	Control	Riesgo residual
I-1	Detalles internos en errores 500	Mensajes genéricos; excepción solo en log (B-6)	Bajo
I-2	XSS almacenado via LLM	`esc()` en todos los `innerHTML` con datos LLM/usuario (B-2)	Bajo
I-3	Secretos en repositorio	gitleaks CI; INC-01 documentado y historial limpiado	Bajo
I-4	Acceso a informes sin autenticar	`GET /reports/download` requiere JWT/Basic	Bajo
I-5	Headers HTTP revelan stack	`SecurityHeadersMiddleware` (B-3)	Bajo

D — Denial of Service

ID	Amenaza	Control	Riesgo residual
D-1	Abuso de `/translate` agota cuota LLM	Rate limiting + auth obligatoria	Medio — sin cuota explícita en código
D-2	PDF gigante agota memoria	Magic bytes + límite 20 MB + límite páginas (B-9)	Bajo
D-3	Bucle LLM sin terminar	Timeout HTTP en SDK	Medio — sin circuit breaker
D-4	Escaneo masivo de activos internos	Validación alcance DNS + `is_global` + allowlist (B-8)	Bajo

E — Elevation of Privilege

ID	Amenaza	Control	Riesgo residual
E-1	Proceso app como root en contenedor	`USER rosetta` (uid 10001), sin `CAP_NET_ADMIN`	Bajo
E-2	Nmap/Nuclei con privilegios ampliados	TCP connect fallback (sin NET_RAW)	Bajo
E-3	Inyección de comandos en parámetros de escaneo	Parámetros como lista; Pydantic valida tipos	Bajo

Amenazas específicas de LLM

ID	Amenaza	Control	Riesgo residual
LLM-1	Prompt injection directa en hallazgo	Tool-use forzado con esquema Pydantic	Medio
LLM-2	XSS via salida del LLM	`esc()` antes de `innerHTML` (B-2)	Bajo
LLM-3	Economic DoS (agotamiento cuota API)	Rate limiting + auth; sin cuota explícita en código	Medio
LLM-4	Alucinación de controles	RAG sobre corpus verificado; tool-use limita IDs	Medio — sin verificador automático de IDs

ID	Amenaza	Control	Riesgo residual
LLM-5	Corpus poisoning via PDF malicioso	PDF extrae hallazgos, no modifica corpus ChromaDB	Bajo

7.3 Incidente INC-01 — Credenciales escritas en un repositorio público

Campo	Valor
Severidad	Alta
Estado	Contenido y corregido
Afecta a	RNF-06, RF-15

Qué pasó. Para dar acceso al profesor se añadió una cuenta `ROSETTA_USERS_EXTRA`. Su contraseña real se escribió como literal en `tests/test_auth.py` (commit `0fafee1`, 2026-05-30) y se publicó en GitHub.

Contención (2026-10-04):

1. Contraseñas nuevas para la cuenta principal y la del profesor, generadas con el módulo `secrets` de Python.
2. Nuevo `ROSETTA_JWT_SECRET` de 64 caracteres.
3. Reinicio del contenedor `app` con `docker compose up -d --no-deps --no-build --force-recreate app`.
4. Verificación: `POST /auth/login` con credenciales antiguas → 401; con nuevas → 200.

Corrección del árbol de trabajo (antes de publicar): los dos commits locales con el literal se sanearon antes del push (`5eb5abd`, `4942403`). `_shots.py` lee credenciales de variables de entorno.

Limpieza del historial (2026-10-06): `git filter-repo --replace-text` sobre un clon `--mirror` temporal. Verificación: `git log -p --all` — 0 apariciones del literal; gitleaks (historial completo, `fetch-depth: 0`) — 0 hallazgos; CI en verde tras force-push (`f50957f` → `4aeef3b`).

El commit original `e2819f0` sigue siendo accesible en GitHub por su hash (HTTP 200 confirmado el 2026-10-06): objeto huérfano que GitHub no elimina automáticamente. **Pendiente:** purga solicitada a GitHub Support (sin respuesta aún).

Lección: las credenciales de prueba siempre salen de variables de entorno o son ficticias; toda credencial compartida con terceros se rota al terminar el uso.

7.4 Riesgos aceptados

R-01 — Vulnerabilidades de chromadb 1.5.9 sin parche publicado

Cuatro CVEs (CVE-2026-45829, CVE-2026-45830, CVE-2026-45831, CVE-2026-45833) afectan al modo servidor HTTP de ChromaDB (RCE, RBAC bypass, acceso cross-tenant). ROSETTA usa ChromaDB embebido (`PersistentClient`, no HTTP server): los CVEs no aplican. El volumen `rosetta_chroma` no tiene puertos publicados. El CI ignora explícitamente estos cuatro identificadores; cualquier otro CVE nuevo rompe el CI.

7.5 Lo que no está hecho

- **Trivy (B-11):** análisis de vulnerabilidades en la imagen Docker. No implementado en este sprint.
- **No hay roles (RBAC):** todos los usuarios autenticados tienen los mismos permisos. No hay distinción entre administrador y auditor de solo lectura.

7.6 Datos personales

Dato	Finalidad	Conservación	Protección
Cuentas de usuario (usuario + hash bcrypt)	Autenticar acceso a la plataforma	Indefinida (mientras exista el `.env`)	`.env` con permisos 600; nunca versionado
Hallazgos y activos detectados	Cumplimiento y trazabilidad normativa	Indefinida en SQLite (`session_store`)	Dentro del contenedor; acceso solo con auth
Contenido de PDFs ingestados	Extraer hallazgos de informes existentes	No se almacena el PDF; solo los hallazgos extraídos	HTTPS en un despliegue tras nginx (en local, solo `localhost`); sin persistencia del PDF original

Envío al LLM: con `LLM_PROVIDER=claude` u `openai`, el contenido del hallazgo y los fragmentos del corpus se envían a la API del proveedor (Anthropic, Groq u otro compatible; HTTPS, fuera del equipo). Con `LLM_PROVIDER=ollama`, todo el procesamiento es local. Para clientes que exijan soberanía de datos completa (ENS, NIS2), la recomendación es `LLM_PROVIDER=ollama`.

Demos: todas las demos y tests usan datos ficticios (empresa TechServ S.A., dominios `.example`, IPs de documentación RFC 5737). Ningún hallazgo real se incluye en el repositorio.

7.7 Despliegue en servidor propio

Existe un despliegue en un servidor propio, heredado de la P1, que a fecha de entrega no está operativo; la evaluación se hace en local con Docker siguiendo el README. Mientras funcionó exponía solo nginx (80 y 443, TLS con Certbot); la app escuchaba en `127.0.0.1:8000` y Neo4j, ChromaDB y SQLite no tenían puertos publicados (B-4: solo se confía en el proxy `172.18.0.1`).

El 2026-10-14 se actualizó a `a7d1abd`, pero `/translate` responde 500 porque la clave del proveedor (Groq) ya no es válida, y el dominio no está en DNS. Para no dejar expuesto un servicio que no se usa, sus contenedores se pararon el mismo día con `docker compose stop`, sin borrar volúmenes, imágenes ni copias; nginx responde 502 para ese nombre. Detalle en `docs/P3_SEGURIDAD.md`, apartado 7.

8. Pruebas y evidencias

8.1 Plan de pruebas

Tipo	Herramienta	Cobertura objetivo
Unitarias	pytest + pytest-asyncio	80 % core, 60 % adapters (objetivo)
API / integración	pytest + httpx AsyncClient	Todos los endpoints en `tests/test_api.py`
Seguridad	`tests/test_security.py`	XSS, path traversal, cabeceras, rate limiting, PDF
E2E en navegador	Docker + Ollama + scripts locales	Golden path: login → traducir → descargar dosier
Instalación limpia	README + clon fresco	RNF-05
Evaluación cuantitativa	`eval/run_eval.py`	F1, hallucination rate, discrepancy rate
Benchmark de latencia	`eval/bench_latency.py` (20 llamadas)	RNF-09: < 30 s; p50, p95 y máximo
CI continua	GitHub Actions	Lint, types, tests, secrets, CVEs

8.2 Resumen de resultados

De las 38 pruebas definidas (tabla completa en Anexo E):

Estado	Cantidad	Pruebas
✔ Pasado	35	PR-01..PR-05, PR-07, PR-08, PR-10..PR-34, PR-36..PR-38
◐ Condicionado	3	PR-06 (solo lab. local), PR-09 (aprueba 23 de 51 traducciones malas), PR-35 (1/20 excede 30 s: arranque en frío)

8.3 Pruebas con resultado no ideal y lección aprendida

Problema	Impacto	Resolución
CI en rojo desde mayo 2026 (RNF-10)	Builds no reproducibles; confianza cero en CI	`setup-uv v7.6.0` + SHA fijados; `uv sync --locked` (commit `2e7792d`)

Problema	Impacto	Resolución
Tests que dependían del entorno (RF-03, RF-17)	Pasaban en local, fallaban en CI sin Docker	Mocks de ChromaDB y Neo4j en CI; tests de integración solo con flag `--integration`
140 vulnerabilidades en pip-audit (RNF-12)	CI auditaba la herramienta, no el proyecto	9 tandas de actualización; solo 4 CVEs de chromadb sin parche quedan como R-01 (commit `9a611d7`)
59,9 s en RNF-09 (1 llamada de 20)	Excede el umbral de 30 s	Es la primera llamada con el modelo fuera de memoria: 32,9 s de carga en la GPU y 20,2 s de la primera evaluación del prompt. En caliente el máximo es 11,0 s y crece con los tokens generados (~59 tokens/s). 0 avisos `marco_desconocido` en las 20 llamadas: la explicación del benchmark D-2 (5 llamadas, «generación verbosa») no se sostiene. Sin cambio de código
B-4 proxies ampliados	Rate limiting evitable desde red interna	Eliminados rangos `10.0.0.0/8` y `172.17.0.0/16` del default (commit `f06f726`)
Swagger UI en blanco (RF-13), visto al capturar esta memoria	`/docs` sin interfaz: la CSP de B-3 bloqueaba cdn.jsdelivr.net	CSP propia para `/docs` y `/redoc`; test E2E con Playwright (`c78a0c7`)
Prompts recortados por Ollama (RF-10, RF-01)	Con la ventana por defecto (4096), el Copilot (5099 tokens) y las traducciones contra los 7 marcos (5655) se recortaban a 2050: respuesta sobre XSS a una pregunta sobre MFA y traducciones sin llamada a la herramienta	`num_ctx` en cada llamada, `OLLAMA_NUM_CTX` = 16384 por defecto (`c56d54c`). El eval y el benchmark RNF-09 usan 1 marco (~1200 tokens) y no se recortaban: sus cifras siguen siendo válidas
Modo Auditoría sin hallazgos contra el laboratorio (RF-06)	Nuclei descartaba el host tras errores de DNS con el nombre interno de Docker; Nmap recibía la URL y escaneaba 0 hosts	Nuclei sin descarte de host, sin interactsh y con tags y severidades configurables; Nmap recibe el host (`b95505f`). Resultado: 18 hallazgos
Estado por marco mezclaba marcos (RF-19)	Con ISO 27001 aparecían controles de PCI-DSS, RGPD y ENS	Índice control → marco construido desde el corpus (`e244421`). Los datos de demo ya guardados en Neo4j hay que regenerarlos
`GET /audit/{id}` respondía siempre 422 (RF-06)	El estado de una auditoría no se podía consultar por REST	Parámetro `request` tipado como `Request` (`3286ed8`)
Textos de la interfaz escritos a mano (RF-12)	El dashboard decía «Claude Sonnet 4.6» y «max 200 MB» con Ollama y un límite de 20 MB	`GET /config/ui` con el LLM en uso, los marcos y `ROSETTA_PDF_MAX_SIZE_MB` (`5b2a3c1`)
Panel Roadmap vacío (RF-20b), encontrado en el ensayo del vídeo del 15/10/2026	El panel «Roadmap Vuln.» no listaba ningún hallazgo: pide `estado=all` y `GET /vuln-roadmap` no reconocía ese valor	El endpoint acepta `estado=all`, con un test en `tests/test_api_controls.py` (`81e76fa`)

8.4 Instalación manual por persona ajena

Un amigo del autor, ajeno al proyecto, lo instaló siguiendo el README el 15/10/2026, sin incidencias según su feedback (PR-31 ✔, RNF-05).

8.5 Evaluación cuantitativa del Traductor

Metodología:

El harness `eval/run_eval.py` pasa cada caso al Traductor llamando directamente a `TraductorSimbiotico.traducir()` (el mismo código que atiende `POST /translate`, sin HTTP), compara los controles ISO 27001:2022 devueltos con el ground truth del profesor (SoA TechServ, `eval/ground_truth/ens_iso.json`) y calcula Precision, Recall y F1 por caso y por familia ENS.

Distinción alucinación vs. discrepancia:

- **Alucinación:** el modelo devuelve un identificador de control que no existe en el estándar ISO 27001:2022. Cualitativamente peor que la discrepancia.
- **Discrepancia:** el modelo devuelve un control real (existe en ISO 27001:2022) pero diferente al que el profesor asignó en el SoA. Es un desacuerdo de interpretación, no una invención.

Resultados (ejecución 2026-10-04):

Modo correspondencia (ENS → ISO 27001:2022)

Métrica	Valor
Casos evaluados	73 / 73
F1 (macro avg)	**0,2648**
Precision (macro avg)	0,4078
Recall (macro avg)	0,2164
Tasa de alucinación	**0,0 %**
Tasa de discrepancia	62,5 %

Modo hallazgo (hallazgos técnicos → ISO 27001:2022)

Métrica	Valor
Casos evaluados	16 / 16
F1 (macro avg)	**0,2813**
Precision (macro avg)	0,3125
Recall (macro avg)	0,2812
Tasa de alucinación	**0,0 %**

Métrica	Valor
Tasa de discrepancia	75,7 %

Validador sobre los 73 casos de correspondencia (informe `2026-10-11T15-28-50`)

Métrica	Traductor solo	Traductor + Validador (solo las aprobadas)
Traducciones entregadas	73	39 (53 %)
Precision (macro avg)	0,4068	0,5222
Recall (macro avg)	0,2205	0,2966
F1 (macro avg)	0,2726	**0,3615**

Veredicto del Validador	F1 ≥ 0,5 (22 casos)	F1 < 0,5 (51 casos)
Aprobada (39)	16	23
Rechazada (34)	6	28

F1 medio de las rechazadas: 0,1706. Precisión de rechazo @F1<0,5: **0,82** (28 de 34). El piloto de 10 casos del 2026-10-04 daba 1,0, pero con solo 3 rechazos.

El Validador no cambia la traducción: la aprueba o la rechaza. Discrimina (las aprobadas tienen el doble de F1 medio que las rechazadas), pero no basta como filtro: aprueba 23 de las 51 traducciones con F1 < 0,5 y rechaza 6 de las 22 con F1 ≥ 0,5. Quedarse solo con las aprobadas sube el F1 de 0,27 a 0,36 a cambio de descartar el 47 % de las traducciones. El F1 del Traductor en esta ejecución (0,2726) difiere del de la ejecución de referencia sin Validador (0,2648, 2026-10-04) sobre los mismos 73 casos porque el LLM no es determinista: el cliente de Ollama no fija temperatura ni semilla.

Limitaciones del eval:

- Solo con `LLM_PROVIDER=ollama` (qwen2.5:14b). Con proveedores en la nube (Claude o compatibles con OpenAI) no se ha ejecutado, por coste; solo hay una prueba de 8 hallazgos con Groq (README).
- Los 16 casos de hallazgo fueron **diseñados por Claude Code** (decisión automatizada en bloque C), no revisados uno a uno por el autor. Sus asignaciones de control son provisionales y los resultados de esos 16 casos son orientativos. El 0 % de alucinación no depende de ellas: se comprueba contra la lista oficial de controles del Anexo A.
- Solo se evalúan ENS→ISO. Los otros cinco marcos (NIS2, DORA, RGPD, NIST CSF, PCI-DSS) no tienen ground truth disponible aún.

9. Matriz de trazabilidad

Requisito	Descripción	Estado final	Implementación	Prueba	Evidencia
RF-01	Traducción hallazgo → controles con cita	✔	`src/rosetta/core/traductor.py` · `POST /translate` · `92d4b83`	PR-01	Vídeo 06:00 · Apdo. 6.1
RF-02	Multi-marco 7 frameworks	✔	`src/rosetta/core/rag.py` · `corpus/` (7 dirs) · `f29cbde`	PR-02	Vídeo 06:00 · Apdo. 6.2
RF-03	RAG ChromaDB 229 frags	✔	`src/rosetta/core/rag.py` · `f29cbde`	PR-03	Vídeo 06:00 · 229 frags en la salida de `load-corpus` · Apdo. 6.2
RF-04	Dosier MD+PDF descargable	✔	`src/rosetta/core/report_generator.py` · `GET /reports/download/{filename}` · `60d437a`	PR-04	Vídeo 07:01 · Apdo. 6.3
RF-05	Ingesta PDF hallazgos	✔	`src/rosetta/core/pdf_ingestion.py` · `POST /ingest/pdf` · `b64dd2b`	PR-05	Vídeo 08:01 · Apdo. 6.4
RF-06	Modo Auditoría RT + WebSocket	◐ Solo lab. local	`src/rosetta/core/orchestrator.py` · `POST /audit/start` · `WS /audit/ws/{id}` · `419b737`	PR-06	Vídeo 10:31 · Apdo. 6.5
RF-07	Ingesta Wazuh JSON/CSV	✔	`src/rosetta/adapters/blue/wazuh.py` · `POST /blue/ingest` · `b64dd2b`	PR-07	Vídeo 10:20 · Apdo. 6.6
RF-08	Correlación Red↔Blue	✔	`src/rosetta/core/blue_enrichment.py` · `b64dd2b`	PR-08	Vídeo 10:20 · 96 % cobertura · Apdo. 6.6
RF-09	Multi-agente Validador	◐ 73 casos: precisión de rechazo 0,82	`src/rosetta/agents/orchestrator.py` · `src/rosetta/agents/validator.py` · `POST /translate` con `"validar": true` · `4ebeade`	PR-09	Vídeo 12:42 · Apdo. 6.7
RF-10	Copilot normativo	✔	`src/rosetta/core/copilot.py` · `POST /copilot/ask` · `b64dd2b`	PR-10	Vídeo 13:24 · Apdo. 6.8
RF-11	Procedure drift	✔	`src/rosetta/core/drift.py` · `POST /drift/analyze` · `b64dd2b`	PR-11	No aparece en el vídeo (por tiempo) · PR-11 · Fig. 8 · Apdo. 6.9
RF-12	Dashboard 10 pan. (base) · 16 (P3)	✔ superado	`src/rosetta/api/dashboard.py` · `GET /dashboard` · `92a5989`	PR-12	Vídeo 03:55 · Apdo. 6.10
RF-13	OpenAPI / Swagger UI	✔	`src/rosetta/api/main.py` (FastAPI auto)	PR-13	Vídeo 14:28 · `GET /openapi.json` → 30 operaciones · Apdo. 6.16 / fig. 12
RF-14	Gate CI/CD diff analyzer	✔	`src/rosetta/core/diff_analyzer.py` · `POST /analyze-diff` · `.github/workflows/rosetta-gate.yml` · `b64dd2b`	PR-14	No aparece en el vídeo (por tiempo) · PR-14 · Apdo. 6.15

Requisito	Descripción	Estado final	Implementación	Prueba	Evidencia
RF-15	Auth JWT + Basic + rate limiting	✔	`src/rosetta/api/auth.py` · `POST /auth/login` · `5cf5aac`	PR-15	No aparece en el vídeo (por tiempo) · PR-15 · Apdo. 6.11
RF-16	Historial SQLite append-only	✔	`src/rosetta/core/session_store.py` · `b64dd2b`	PR-16	Vídeo 15:09 · 82 % cobertura · Fig. 3
RF-17	Grafo Neo4j + degradación	✔	`src/rosetta/core/graph.py` · `GET /graph/data` · `1c567c4`	PR-17	Vídeo 16:31 · Apdo. 6.12
RF-18	CLI 6 comandos	✔	`src/rosetta/cli/main.py` · `06bffda`	PR-18	No aparece en el vídeo (por tiempo) · PR-18 · Apdo. 6.13
RF-19	Panel cumplimiento	✔	`src/rosetta/api/main.py` · `GET /compliance/state/{marco}` · `b64dd2b`	PR-19	Vídeo 17:53 · 100 % cobertura · Apdo. 6.14 / fig. 11
RF-20a	Catálogo controles	✔	`src/rosetta/core/control_store.py` · `GET /controls/{marco}` · `GET /controls/{marco}/{control_id}` · `b64dd2b`	PR-20	Vídeo 17:56 · Apdo. 6.14
RF-20b	Roadmap vulns	✔	`src/rosetta/api/main.py` · `GET /vuln-roadmap` · `b64dd2b` · `81e76fa`	PR-21	Vídeo 17:57 · Apdo. 6.14
RF-20c	Panel evidencias	✔	`src/rosetta/api/main.py` · `GET /evidence-panel` · `b64dd2b`	PR-22	Vídeo 17:59 · Apdo. 6.14
RF-20d	Gap analysis	✔	`src/rosetta/api/main.py` · `POST /gap-analysis/{marco}` · `b64dd2b`	PR-23	Vídeo 19:26 · Apdo. 6.14
RF-20e	Plan director	✔	`src/rosetta/api/main.py` · `POST /plan-director/{marco}` · `b64dd2b`	PR-24	Vídeo 19:53 · Apdo. 6.14
RF-20f	Análisis riesgos	✔	`src/rosetta/api/main.py` · `POST /risk-analysis` · `b64dd2b`	PR-25	Vídeo 19:54 · Apdo. 6.14
RNF-01	Degradación grácil Neo4j	✔	`src/rosetta/core/graph.py` (`verify_connectivity`) · `src/rosetta/api/main.py` (`neo4j_unavailable_api_continues`) · `1c567c4`	PR-26	Vídeo 22:22 (en directo) · PR-26 · 4 tests degradación · Apdo. 6.12
RNF-02	Cobertura ≥ 80 % en core	✔ 92 % en `core/`	`pytest --cov=rosetta --cov-branch` (CI)	PR-27	Vídeo 23:23 (diapositiva) · PR-27 · 85 % global · `pdf_ingestion.py` 76 %
RNF-03	mypy 0 errores	✔	`mypy src/` (56 ficheros)	PR-28	Vídeo 23:23 (diapositiva) · PR-28 · 0 errores
RNF-04	ruff 0 violaciones	✔	`ruff check . && ruff format --check` · `7ec6b66`	PR-29	Vídeo 23:23 (diapositiva) · PR-29 · CI run 37203612212 verde
RNF-05	Instalación limpia	✔	`Dockerfile` · `docker-compose.yml` · `uv.lock` · `2339813`	PR-30, PR-31	Vídeo 23:23 (diapositiva) · PR-30: build 165 s · PR-31: instalación por persona ajena sin incidencias (15/10/2026) · Apdo. 8.4

Requisito	Descripción	Estado final	Implementación	Prueba	Evidencia
RNF-06	Cero secretos	✔	`.github/workflows/ci.yml` (gitleaks, fetch-depth 0) · `.pre-commit-config.yaml` · `1ef7438` · `4aeef3b`	PR-32	Vídeo 22:55 (diapositiva) · PR-32 · 0 apariciones filter-repo · Apdo. 7.3
RNF-07	Rate limiting	✔	`src/rosetta/api/main.py` (slowapi) · `5cf5aac`	PR-33	Vídeo 21:31 (en directo) · PR-33 · 5 logins por minuto: 5 × 401 y la sexta 429
RNF-08	Alcance Modo Auditoría	✔	`src/rosetta/core/orchestrator.py:_validar_alcance()` · `810812d`	PR-34	Vídeo 21:50 (en directo) · PR-34 · `http://192.168.1.10` → 400
RNF-09	Latencia < 30 s	◐ p50=6,95 s	`eval/bench_latency.py` · `eval/reports/2026-10-11T15-28-12_bench_latency_ollama.json`	PR-35	Vídeo 23:23 (diapositiva) · PR-35 · p50=6,95 s · p95=11,0 s · máx=59,9 s (en frío)
RNF-10	CI reproducible	✔	`.github/workflows/ci.yml` · `uv.lock` · `2e7792d`	PR-36	Vídeo 23:23 (diapositiva) · PR-36 · CI run 37203612212 verde
RNF-11	Cabeceras seguridad HTTP	✔	`src/rosetta/api/main.py` (SecurityHeadersMiddleware) · `c552729`	PR-37	Vídeo 22:04 (en directo) · PR-37 · X-Frame-Options, CSP presentes
RNF-12	CVE audit CI	✔	`.github/workflows/ci.yml` (dependency-audit) · `2e7792d` · `26e63e5`	PR-38	Vídeo 23:23 (diapositiva) · PR-38 · R-01 aceptado; fsspec parcheado

10. Limitaciones y trabajo futuro

10.1 Qué no funciona

- **Commit huérfano `e2819f0` accesible en GitHub:** el objeto persiste tras el filter-repo porque GitHub requiere una solicitud explícita de purga al soporte. Pendiente de respuesta.
- **Error 500 genérico de `POST /translate` cuando falla la clave del proveedor LLM:** con una `ANTHROPIC_API_KEY` no válida, Anthropic responde 401 y la API devuelve «Internal Server Error», sin indicar que el fallo está en el proveedor (visto el 2026-10-14). El cuerpo no filtra datos, pero no ayuda a diagnosticar; lo adecuado sería un 502 o 503 con un mensaje claro. Con el plan gratuito de Groq ocurre lo mismo al superar el límite de tokens por minuto (429); además, tras ese 500 la API cierra la conexión, y `seed_demo.py` se interrumpe en la tercera traducción. No se corrige: el código está congelado desde el 2026-10-14.
- **Despliegue en servidor propio no operativo:** existe un despliegue en un servidor propio que a fecha de entrega no está operativo; la evaluación se hace en local con Docker siguiendo el README. La clave del proveedor LLM (Groq) ya no es válida y el dominio no está en DNS; los contenedores están parados desde el 2026-10-14 (apdo. 7.7).
- **NIS2 en traducciones con varios marcos:** en las pruebas del 2026-10-14 con 3 marcos (ISO 27001, ENS y NIS2) sobre 8 hallazgos, ni `qwen2.5:14b` ni `qwen2.5:7b` citaron ningún

control de NIS2; con Groq (`openai/gpt-oss-120b`) sí aparecieron en los 8, pero con 11 de 41 controles fuera del corpus. Pedir varios marcos no garantiza que la respuesta los cubra todos. La causa (recuperación del RAG o el propio LLM) no se ha investigado.

10.2 Qué funciona con condiciones

- **RF-06 — Modo Auditoría Red Team:** implementado y con tests, pero la demo en el vídeo se hace contra el laboratorio local. Una demo real requeriría un objetivo con autorización explícita escrita.
- **RF-09 — Validador:** disponible con `"validar": true`, pero off por defecto; el scheduler Soundwave no tiene endpoint de producción. Sobre 73 casos, el Validador aprueba 23 de las 51 traducciones con F1 < 0,5: discrimina, pero no sustituye la revisión humana.
- **RNF-09 — Latencia del Traductor:** en 20 llamadas con Ollama qwen2.5:14b, p50 = 6,95 s y p95 = 11,0 s. Solo la primera, con el modelo fuera de memoria, supera los 30 s (59,9 s: 32,9 s de carga y 20,2 s de la primera evaluación del prompt). En caliente el máximo es 11,0 s.

10.3 Qué se haría con más tiempo

- **Roles (RBAC):** distinción entre administrador y auditor de solo lectura.
- **Adaptadores Shodan / HIBP:** el enum `OrigenHallazgo.SHODAN/HIBP` existe, pero no hay adaptador.
- **Eval de los cinco marcos restantes** (NIS2, DORA, RGPD, NIST CSF, PCI-DSS): requieren un SoA externo equivalente.
- **Eval con Claude:** el harness está preparado, pero el coste de 73 traducciones vía API lo hace inviable en fase de desarrollo.
- **Purga GitHub Support** del commit huérfano `e2819f0`: pendiente de respuesta.
- **Trivy (B-11):** escaneo de vulnerabilidades en la imagen Docker.

11. Reparto del trabajo

Proyecto individual con autorización del equipo docente.

Historial con un único autor (Michael Jordan Telleria Guadalajara). Cuenta de GitHub: `MichaelJTG`; todos los commits llevan `Author: Michael <michael.jt.pro@gmail.com>`.

Cronología real del proyecto

Fase	Período real	Actividades principales
P1 (Práctica 1)	2026-04-18 a 2026-05-25 (entrega); último commit 2026-05-30	Diseño de arquitectura, 43 commits, producto completo desplegado en un servidor propio: API, dashboard 10 paneles, RAG 7 marcos, multi-agente, Docker, JWT. Claude Code como asistente de programación en todas las fases.

Fase	Período real	Actividades principales
Fase 0 — auditoría y corrección CI	2026-09-22	Diagnóstico inicial: CI rota, bloqueos activos, estado de tests. Planificación de bloques A–E.
Bloques A–E (P3)	2026-10-04 a 2026-10-06	Implementación de todos los bloques: seguridad B-1..B-11, corpus ENS, eval harness, Docker Nmap/Nuclei, fixes CI, CLI tests, Validador, compliance panels. Realizado mayoritariamente por Claude Code de forma autónoma en sesiones los días 4 a 6 de octubre; dirección y revisión por bloques a cargo del autor.
Limpieza historial (INC-01)	2026-10-06	filter-repo, verificación gitleaks, force-push. Ejecutado por el autor.
Memoria técnica	2026-10-06	Redactada por Claude Code a partir de los documentos `P3_*.md`; revisada y corregida por el autor.

Horas del autor en la P3: al menos 40–50 (estimación propia, sin registro horario).

12. Uso de herramientas de IA

Herramienta	Uso en el proyecto
Claude Code	Asistente de programación principal en P1 (todas las fases, ciclo investigar→diseñar→implementar+tests→revisar→commit) y en P3 (bloques A–E de forma autónoma bajo dirección del autor; redacción de esta memoria a partir de los documentos `P3_*.md`). Los 13 commits co-firmados por Claude Code (del 2026-10-04 al 2026-10-06) indican Claude Sonnet 4.6.
Claude (claude.ai)	Auditoría del repositorio, planificación de bloques de trabajo, revisión de decisiones de arquitectura (ADR), revisión de esta memoria.
LLM del producto	Intercambiable con `LLM_PROVIDER`: Ollama `qwen2.5:14b` en local (`LLM_PROVIDER=ollama`), el proveedor de desarrollo con el que se hicieron todos los evals y el benchmark; Claude (Anthropic) o una API compatible con OpenAI. El despliegue de la P1 usaba Groq (`llama-3.3-70b-versatile`). Con proveedores en la nube no se ha ejecutado el eval cuantitativo; solo una prueba de 8 hallazgos con Groq (`openai/gpt-oss-120b`, README).
Herramientas de IA en P1	Gemini, para planificar y ordenar ideas. Claude Code, para la programación; parte del código lo escribió el autor. Además, los conocimientos del Grado Superior en DAM.

Ground truth del eval: los 73 casos de correspondencia ENS→ISO tienen como ground truth el SoA TechServ facilitado por el equipo docente. **No fue generado por IA.** Los 16 casos de hallazgo técnico fueron **diseñados por Claude Code** (bloque C, decisión automatizada), no revisados uno a uno por el autor. Sus asignaciones de control son provisionales y los resultados de esos 16 casos son orientativos. El 0 % de alucinación no depende de ellas: se comprueba contra la lista oficial de controles del Anexo A.

Declaración del autor: puedo explicar cualquier parte del código, la arquitectura y las decisiones de diseño de ROSETTA. La IA es un asistente de programación, no el autor intelectual del proyecto.

13. Anexos

Anexo A — Manual de instalación ampliado

Requisitos del sistema:

- Docker Desktop (o Docker Engine + Compose) ≥ 24
- Ollama ≥ 0.4 con modelo `qwen2.5:14b` descargado: `ollama pull qwen2.5:14b`
- Python 3.11+ y `uv` (solo para desarrollo; no necesarios para la instalación Docker)

Instalación paso a paso:

```
# 1. Clonar el repositorio
git clone https://github.com/MichaelJTG/Rosetta
cd Rosetta

# 2. Crear el fichero de configuración
cp .env.example .env
# Editar .env: ROSETTA_USER, ROSETTA_PASSWORD (>=12 chars), ROSETTA_JWT_SECRET (>=32
chars)
# Para usar Ollama local: LLM_PROVIDER=ollama
# Para usar Claude (no verificado): LLM_PROVIDER=claude y ANTHROPIC_API_KEY=<tu clave>
# Groq, compatible con OpenAI (probado el 2026-10-14; resultados en el README):
#   LLM_PROVIDER=openai, OPENAI_API_KEY, OPENAI_API_BASE=https://api.groq.com/openai/
v1,
#   OPENAI_MODEL=openai/gpt-oss-120b

# 3. Construir e iniciar
docker compose up -d --build
# Build aprox. 165 s; imagen resultante 3,85 GB

# 4. Cargar el corpus normativo
docker compose exec app rosetta load-corpus all corpus/
# Resultado esperado: 229 fragmentos indexados
# IMPORTANTE: reiniciar la app tras indexar si el proceso lo cargó externamente
docker compose restart app

# 5. Verificar
curl http://localhost:8000/health
# {"status":"ok","version":"0.1.0"}

# 6. Acceder
# Dashboard: http://localhost:8000/dashboard
# API docs:  http://localhost:8000/docs
```

Problemas encontrados durante la instalación limpia (2026-10-04):

1. **Reinicio tras `load-corpus`:** tras indexar el corpus desde `docker compose exec`, la API no ve los fragmentos sin reiniciar. Solución: `docker compose restart app`.

2. **Memoria para Ollama:** el modelo `qwen2.5:14b` ocupa 11,9 GB de RAM o VRAM con la ventana que pide ROSETTA (`OLLAMA_NUM_CTX=16384`); con 4096 baja a 9,5 GB, pero Ollama recorta los prompts largos. Solución: sin esa memoria libre, usar Claude (`LLM_PROVIDER=claude` y `ANTHROPIC_API_KEY`). Un modelo menor no sirve: con `qwen2.5:7b` (5,6 GB), en 8 hallazgos por `POST /translate` (2026-10-14), ningún control ISO coincidió con el ground truth (`qwen2.5:14b`: F1 0,17 en los mismos 8) y, con 3 marcos, 21 de sus 55 controles no estaban en el corpus.
3. **Neo4j en modo degradado sin advertencia:** si Neo4j no arranca, la app arranca igualmente y el grafo funciona en modo memoria. Verificar con `docker compose ps`.

Anexo B — Credenciales de prueba (solo locales y ficticias)

No hay instancia desplegada operativa a fecha de entrega (apdo. 10.1): se evalúa en local, con las credenciales del `.env` de quien instala (Anexo A).

Credenciales de desarrollo local (válidas solo para instalaciones desde `.env.example` sin modificar):

Campo	Valor
Usuario	`auditor-demo`
Contraseña	`cambia-esta-clave-de-prueba`
Método	`POST http://localhost:8000/auth/login` con JSON `{"username":"...","password":"..."}`
Token devuelto	JWT Bearer, incluir en `Authorization: Bearer <token>`

> **Advertencia:** cambiar estas credenciales antes de exponer la aplicación fuera de localhost.

Anexo C — Glosario

Término	Definición
HallazgoMaestro	Modelo Pydantic que representa un hallazgo técnico normalizado: origen, activo detectado, evidencia, vector de ataque, severidad y marco normativo destino.
Traductor Simbiótico	El componente núcleo de ROSETTA: combina RAG (ChromaDB) con un LLM usando tool-use forzado para traducir un hallazgo técnico a controles normativos con cita y justificación.
RAG	Retrieval-Augmented Generation: técnica que inyecta fragmentos relevantes del corpus normativo en el contexto del LLM antes de generar la respuesta.
Tool-use forzado	Configuración del LLM en la que solo puede responder usando una herramienta/función con esquema predefinido (Pydantic), impidiendo texto libre. Reduce alucinaciones al 0 % en los evals.

Término	Definición
Corpus	Conjunto de fragmentos de texto normativo indexados en ChromaDB. Incluye 7 marcos: ISO 27001:2022, ENS RD 311/2022, NIS2, DORA, RGPD, NIST CSF 2.0, PCI-DSS 4.0.
Ground truth	Asignaciones de control normativo correctas para evaluar el Traductor. El ground truth ENS→ISO fue proporcionado por el equipo docente (SoA TechServ); no fue generado por IA.
Alucinación (eval)	El Traductor devuelve un identificador de control que no existe en el estándar. Cualitativamente peor que la discrepancia.
Discrepancia (eval)	El Traductor devuelve un control real pero diferente al del ground truth. Puede deberse a interpretación legítimamente distinta.
Procedure drift	Divergencia entre un procedimiento escrito y el comportamiento real observado por los sensores. ROSETTA cuantifica este drift para apoyar revisiones de procedimientos.
Validador	Segundo agente LLM que revisa la traducción del Traductor y la aprueba o la rechaza. Se invoca con el campo `"validar": true` de `POST /translate`.
Degradación grácil	Capacidad de la plataforma de seguir funcionando cuando un componente no esencial (Neo4j) no está disponible, sin errores visibles para el usuario.
ENS	Esquema Nacional de Seguridad (RD 311/2022): marco normativo de referencia para la seguridad de los sistemas de información en la Administración Pública española.
INC-01	Incidente de seguridad: contraseña real escrita en el repositorio público (commit `0fafee1`, 2026-05-30). Contenido y corregido: credenciales rotadas (2026-10-04) e historial limpiado (2026-10-06).
R-01	Riesgo aceptado: 4 CVEs de chromadb 1.5.9 en modo servidor HTTP. No aplican a ROSETTA (usa ChromaDB embebido).

Anexo D — Referencias

Recurso	Tipo	Uso en ROSETTA
RD 311/2022 (BOE-A-2022-7191)	Normativa	Corpus ENS: 73 medidas del Anexo II
ISO/IEC 27001:2022	Normativa	Marco principal de traducción; 93 controles del Anexo A
CCN-STIC 825	Guía de adecuación	Referencia para el mapeo ENS→ISO en el corpus
Directiva NIS2 (UE 2022/2555)	Normativa	Corpus NIS2
Reglamento DORA (UE 2022/2554)	Normativa	Corpus DORA
RGPD (Reglamento UE 2016/679)	Normativa	Corpus RGPD
NIST CSF 2.0 (NIST CSWP 29)	Marco	Corpus NIST CSF
PCI DSS 4.0 (PCI SSC)	Estándar	Corpus PCI-DSS
FastAPI (tiangolo/fastapi)	MIT	Framework API REST

Recurso	Tipo	Uso en ROSETTA
Pydantic v2	MIT	Validación de modelos y tool-use forzado
ChromaDB 1.5.9	Apache 2.0	Vector store embebido para el RAG
Neo4j Community Edition	GPL 3.0	Grafo de correlación (orquestado, no forkeado)
Ollama	MIT	Runtime LLM local
Anthropic SDK	MIT	Cliente Claude API
Nmap	Licencia Nmap	Escáner de red (orquestado vía CLI)
Nuclei 3.11.1	MIT	Escáner de vulnerabilidades (orquestado vía CLI)
Wazuh	GPL 2.0	SIEM Blue Team (orquestado vía API JSON)
WeasyPrint	BSD	Generación de PDF desde HTML/CSS
sentence-transformers	Apache 2.0	Modelo de embeddings multilingüe
structlog	Apache 2.0	Logs estructurados con audit trail
slowapi	MIT	Rate limiting para FastAPI
uv	MIT	Gestor de paquetes con lockfile determinista
ruff	MIT	Linter y formateador Python
mypy	MIT	Comprobación de tipos estricta
gitleaks	MIT	Escáner de secretos en CI
pip-audit	Apache 2.0	Audit CVE de dependencias

Anexo E — Tabla completa de pruebas (PR-01...PR-38)

ID	Requisito	Descripción	Resultado esperado	Resultado obtenido	Estado
PR-01	RF-01	`POST /translate` con hallazgo válido y corpus cargado	200, `controles_incumplidos` ≥ 1, `cita_normativa` no vacía	200, respuesta en 7,3 s con controles ENS e ISO correctos	✔
PR-02	RF-02	`POST /translate` con `marco=ens`, luego `marco=nist`	Respuesta distinta para cada marco; IDs de control válidos	Controles específicos de ENS en primer caso, NIST CSF en segundo	✔
PR-03	RF-03	`rosetta load-corpus all corpus` y reinicio de la app	Los 7 marcos indexados en ChromaDB	229 fragmentos en la salida del comando (2026-10-10)	✔

ID	Requisito	Descripción	Resultado esperado	Resultado obtenido	Estado
PR-04	RF-04	`POST /reports/generate` + `GET /reports/download/{nombre}.md`	200, fichero MD descargable con hallazgos	MD descargado con estructura correcta; PDF también descargado	✔
PR-05	RF-05	`POST /ingest/pdf` con PDF de ejemplo TechServ	Lista de hallazgos extraídos	3 hallazgos extraídos del PDF de ejemplo	✔
PR-06	RF-06	`POST /audit/start` + `WS /audit/ws/{id}` contra laboratorio local	WebSocket emite progreso; hallazgos almacenados	Nmap y Nuclei ejecutan; progreso visible en dashboard; 18 hallazgos contra el laboratorio (2026-10-10)	◐ Solo laboratorio local
PR-07	RF-07	`POST /blue/ingest` con JSON de alertas Wazuh	`DatosBlue` normalizado, 200	Alertas normalizadas correctamente	✔
PR-08	RF-08	`BlueEnrichment.enriquecer()` con hallazgos Red + alertas Blue	Hallazgos enriquecidos con correlación	Cruce correcto por activo; 96 % cobertura	✔
PR-09	RF-09	`POST /translate` con `"validar": true`	Respuesta con campo `validacion` del Validador	73 casos: aprobadas F1 0,36, rechazadas 0,17; precisión de rechazo 0,82	◐ aprueba 23 de 51 traducciones malas
PR-10	RF-10	`POST /copilot/ask` con pregunta sobre ENS	Respuesta con `confianza` ≥ 0 y citas del corpus	Cita de ENS `org.2` correcta; confianza 0,82	✔
PR-11	RF-11	`POST /drift/analyze` con procedimiento y hallazgos	`drift_score` y lista de diferencias	Score 0,45 con 2 diferencias detectadas	✔
PR-12	RF-12	`GET /dashboard`	HTML con los 16 paneles	200; los 16 paneles visibles en navegador	✔
PR-13	RF-13	`GET /openapi.json` y `GET /docs`	Todas las rutas REST salvo `/dashboard` en el esquema; Swagger UI visible	29 rutas y 30 operaciones en el JSON; Swagger UI carga en el navegador (`tests/e2e/test_docs_ui.py`, 2026-10-10)	✔
PR-14	RF-14	`POST /analyze-diff` con diff ficticio que añade endpoint sin auth	`decision=block`	`decision=block`; mensaje con control incumplido	✔

ID	Requisito	Descripción	Resultado esperado	Resultado obtenido	Estado
PR-15	RF-15	`POST /auth/login` con credenciales incorrectas 6 veces	401 en los 5 primeros intentos y 429 en el sexto (límite de 5 logins por minuto)	`401 401 401 401 401 429` (ensayo del 2026-10-15)	✔
PR-16	RF-16	`POST /translate` × 3; `GET /findings`	3 registros en historial	3 hallazgos en SQLite con timestamps correctos	✔
PR-17	RF-17	`GET /graph/data` con Neo4j apagado	200 con grafo en memoria (no 500)	200 con nodos y aristas en memoria	✔
PR-18	RF-18	`rosetta version` en terminal	Semver (p.e. `1.0.0`)	`1.0.0` impreso en stdout	✔
PR-19	RF-19	`GET /controls/iso27001`	Controles agrupados con estado	Lista de controles ISO con estado cumplimiento	✔
PR-20	RF-20a	`GET /controls/ens/org.2`	Detalle del control con estado y hallazgos	Detalle correcto; estado `pendiente` por defecto	✔
PR-21	RF-20b	`GET /vuln-roadmap`	Lista de hallazgos con trazabilidad temporal	Hallazgos ordenados por fecha con marco asociado	✔
PR-22	RF-20c	`GET /evidence-panel`	Evidencias ligadas a controles	Evidencias con enlace al hallazgo fuente	✔
PR-23	RF-20d	`POST /gap-analysis/iso27001`	Brechas con severidad	Controles sin cobertura con severidad estimada	✔
PR-24	RF-20e	`POST /plan-director/iso27001`	Plan priorizado	Lista de acciones ordenadas por prioridad	✔
PR-25	RF-20f	`POST /risk-analysis`	Activos con scoring	Activos con score calculado a partir de severidad y exposición	✔
PR-26	RNF-01	Arrancar con `NEO4J_URI=bolt://localhost:9999` (Neo4j inaccesible)	App arranca; todos los endpoints 200	App arranca en 3 s; `GET /graph/data` → 200 con grafo vacío	✔

ID	Requisito	Descripción	Resultado esperado	Resultado obtenido	Estado
PR-27	RNF-02	`uv run pytest --cov=rosetta --cov-branch`	Cobertura ≥ 80 % en `core/`	85 % global y 92 % en `core/` (2026-10-10); `pdf_ingestion.py` 76 %	✔
PR-28	RNF-03	`mypy src/`	0 errores	0 errores en 56 ficheros	✔
PR-29	RNF-04	`ruff check . && ruff format --check`	0 violaciones	CI run 37203612212 verde	✔
PR-30	RNF-05	Instalación limpia desde cero	App funcional en < 10 min	Build 165 s; 197 fragmentos indexados (corpus anterior a la ampliación del ENS); login OK	✔
PR-31	RNF-05	Instalación por persona ajena al proyecto	Instalación completa siguiendo solo el README	Un amigo del autor, ajeno al proyecto, lo instaló siguiendo el README el 15/10/2026, sin incidencias según su feedback	✔
PR-32	RNF-06	`git log -p --all | grep <literal>`	0 apariciones	0 apariciones tras filter-repo (2026-10-06)	✔
PR-33	RNF-07	`POST /auth/login` × 6 con IP fija	Límite de 5 logins por minuto: 5 respuestas 401 y la sexta 429	`401 401 401 401 401 429` (ensayo del 2026-10-15)	✔
PR-34	RNF-08	`POST /audit/start` con objetivo `http://192.168.1.10`	Error de alcance	400 `{"detail":"El objetivo 'http://192.168.1.10' apunta a una dirección privada/reservada y no puede ser escaneado. Para un laboratorio autorizado, añádelo a ROSETTA_AUDIT_ALLOWLIST en el servidor."}`	✔
PR-35	RNF-09	Benchmark: 20 traducciones con Ollama qwen2.5:14b (`eval/bench_latency.py`)	< 30 s	p50=6,95 s · p95=11,0 s · máx=59,9 s	◐ 1/20 excede 30 s (en frío)
PR-36	RNF-10	CI verde con Python 3.11 y 3.12	Build en verde	Run 37203612212 verde	✔

ID	Requisito	Descripción	Resultado esperado	Resultado obtenido	Estado
PR-37	RNF-11	`GET /health` → inspeccionar cabeceras	`X-Frame-Options: DENY`, CSP, `Referrer-Policy`	Presentes en todas las respuestas	✔
PR-38	RNF-12	CI job `dependency-audit`	0 CVEs nuevos	4 chromadb (R-01 aceptado); fsspec CVE parcheado	✔